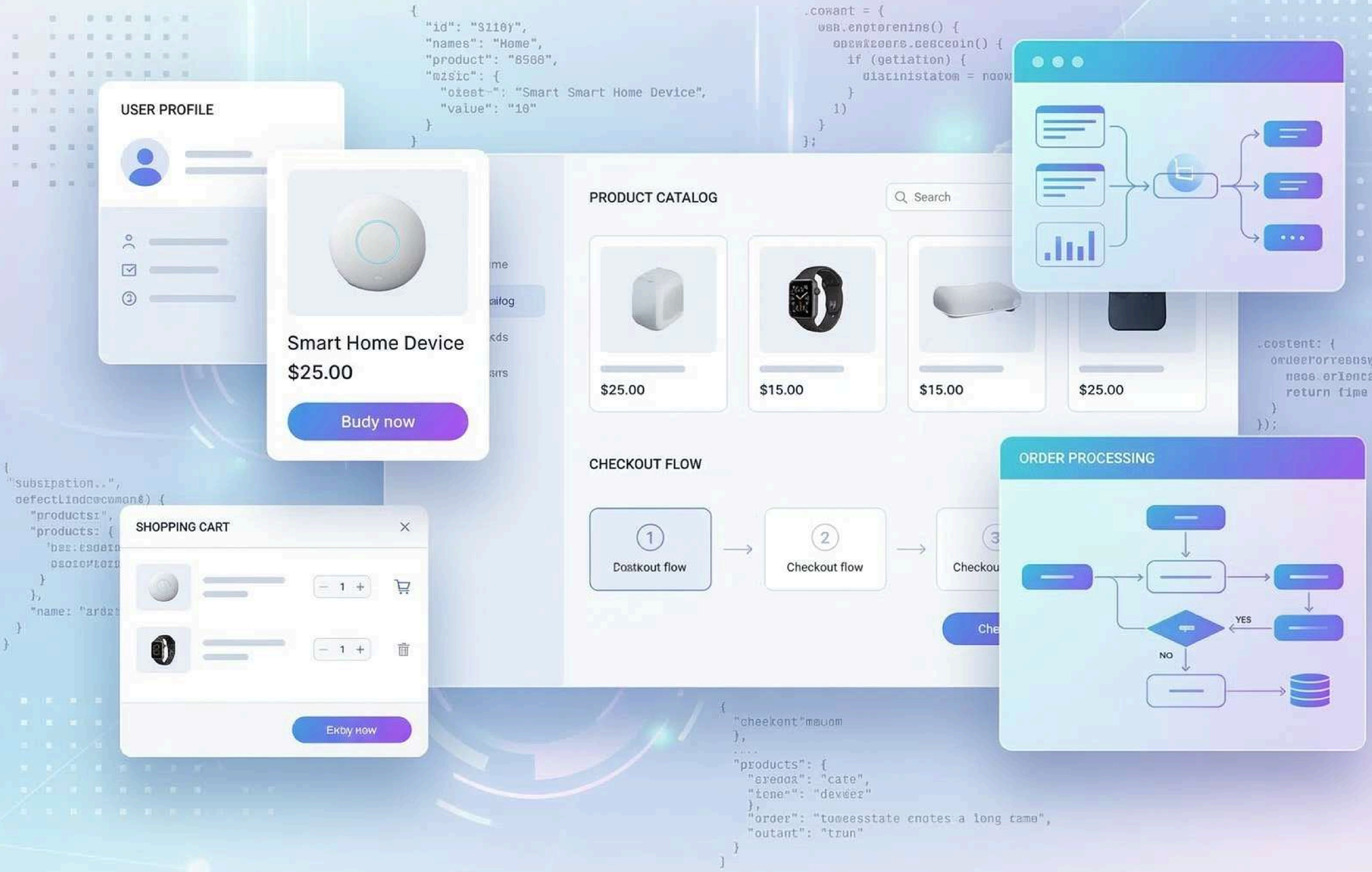
Marketly



Team Members
Sohaib Ayman Elsayed Elbadawy Ashry
Hossam Hassan Mostafa Hassan
Mohamed Ahmed Thabet Hussein
Rawan Hamdi Mohamed Saad
Abdelrahman Khaled Slahelden Mohamed

Agenda

Problem Definition :

With the rapid development of online shopping, users need to find an easier way to shop online, such as browsing products, managing their selected products, and shopping online via an online platform. Many shopping activities involve physical movements to stores, which can be tedious.

Moreover, some online shopping platforms may not provide clear product management, online navigation between product categories, or an efficient shopping cart and checkout process. Users may face some issues while shopping online, such as product management or shopping online.

Therefore, there is a need to find an easier way for users to shop online via an online platform, such as browsing products, product management, shopping cart management, and shopping online via the web.

Proposed Solution :

To overcome the problems associated with the conventional shopping methods and ineffective online shopping experiences, the project aims at developing an e-commerce website through which users can easily browse and purchase products using an effective and user-friendly interface.

The project provides various categories of products through which users can easily find the products they want to purchase. It also provides a search option through which users can search for a particular product. It also provides a dynamic shopping cart through which users can easily manage the products they want to purchase.

The project also provides a login facility through which users can restrict certain operations, such as the checkout process. It also provides a dark mode option through which users can have a good user interface with the system.

The project aims at developing an effective and user-friendly interface through which users can have a good shopping experience.

Existing E-Commerce Platforms :

To better understand our system, we decided to compare it to other popular e-commerce platforms, such as Amazon and eBay. They are popular across the world and are known for their advanced e-commerce services. By studying these platforms, we can better understand the common features of today's e-commerce platforms.

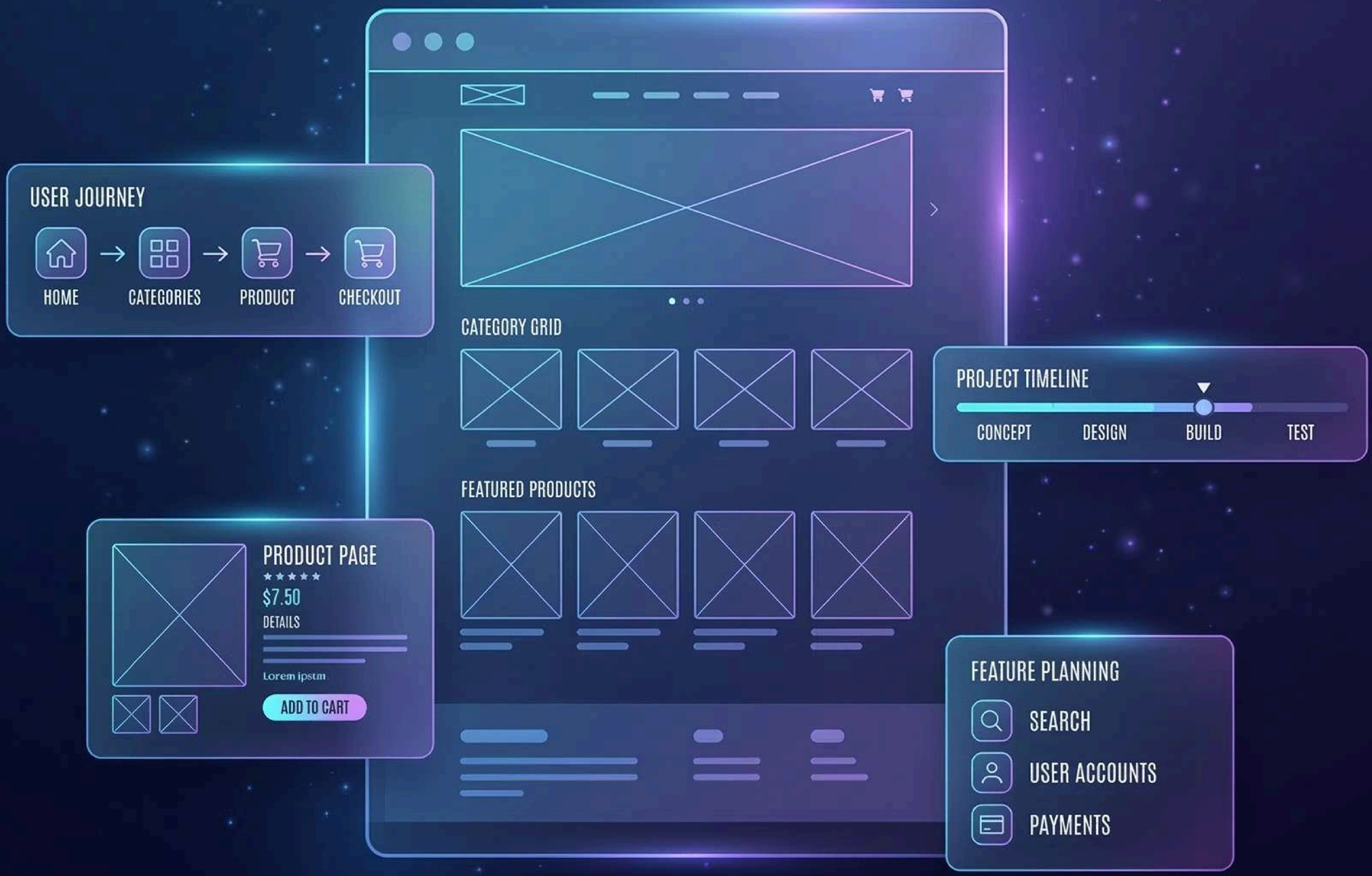
Amazon is one of the most popular e-commerce platforms across the world. Users can find millions of products across different categories, such as electronics, clothes, books, and many more. Users can use the platform to search for products, view product details, use the shopping cart, and check out products. At its core, Amazon is designed to provide an efficient shopping experience to its users.

eBay is another popular e-commerce platform across the world. Users can use this platform to buy and sell products online. Users can find products across different categories, view product details, and use the shopping cart to purchase products. eBay is popular for its auction-based business model, where users can bid for products instead of buying them.

System Comparison Table

Feature	Amazon	eBay	Marketly
User Authentication	True	True	True
Product Categories	True	True	True
Product Search	True	True	True
Product Details Page	True	True	True
Dynamic Search Cart	True	True	True
Cart Preview on Hover	False	False	True
Cart Item Counter	True	True	True
Persistent Cart	True	True	True
Price Filter	True	True	True
Price Sorting	True	True	True
Dark Mode	False	False	True
Toast Notfications	False	False	True
Responsive Design	True	True	True

PLANNING


USER JOURNEY
HOME
CATEGORIES
PRODUCT
CHECKOUT
CATEGORY GRID
FEATURED PRODUCTS
PROJECT TIMELINE
CONCEPT
DESIGN
BUILD
TEST
PRODUCT PAGE
$7.50
DETAILS
Lorem ipstm
ADD TO CART
FEATURE PLANNING
SEARCH
USER ACCOUNTS
PAYMENTS

1.1 System Request

Project Sponsor: Marketly Team.

Business Need:

- Traditional online shopping systems often suffer from poor usability, limited product categorization, and weak cart management. NextGen addresses these gaps by:
- Offering category-based product browsing and search for faster product discovery
- Implementing a dynamic, persistent shopping cart that improves purchase flow
- Ensuring secure authentication and login validation for all users
- Providing a scalable system that can grow with increasing product listings and user traffic

Business Requirements:

- User & Authentication:
  - User registration and login validation
  - Login required for checkout
  - Persistent user sessions
- Product Management:
  - Product categorization and category-based browsing
  - Product search within categories
  - Price filtering and sorting options
  - Product detail pages with image galleries
- Shopping Cart & Checkout:
  - Dynamic shopping cart with item counter
  - Cart preview dropdown for quick access
  - Cart management (Add / Remove / Update Quantity)
  - Persistent cart using LocalStorage
  - Secure checkout system
  - Order confirmation and success page
- UI & UX Features:
  - Quantity selector for products
  - Toast notifications for actions
  - Dark mode toggle
  - Responsive design for desktop and mobile
  - Scroll-to-top navigation button

Business Value:

The platform generates revenue through multiple streams:

1. Direct Product Sales – revenue from product purchases
2. Featured Product Advertisements – sellers pay for promotional placement
3. Premium Listings – additional fees for priority display on category pages

Estimated monthly revenue: $2,500–$3,500 (based on sample order volume and advertising projections)

Special Issues or Constraints:

- The system should take into account several key constraints and challenges during development and utilization:
- User Data Security:
  - The system should guarantee the security of user data and prevent unauthorized access to such data.
- System Performance Under High Load:
  - The system should guarantee good performance even when several users are utilizing the system.
- Responsive Design (Mobile Support):
  - The system should be fully responsive to allow utilization of the system by different devices such as mobile phones and tablets.

1.2 Feasibility Analysis

Technical Feasibility :

The project can be developed using modern frontend technologies such as HTML, CSS, JavaScript and React.
The team is familiar with the technologies required to build the project, including HTML, CSS, JavaScript, and React.
These technologies are suitable for creating an interactive frontend website.
Product data will be imported from the Platzi API through a seeding process and stored in Cloud Firestore, then displayed dynamically on the website.
The shopping cart will be managed using Redux Toolkit, while application data such as products, categories, and users will be managed using React Context and Cloud Firestore.
Since the technologies are widely used and well-documented, the project is technically achievable and suitable for a frontend development team.
The project size is manageable for a frontend development team since it includes product listing, cart functionality, real order processing, order tracking, user management, product management, and administrative dashboard features.

Economic Feasibility :

The development cost of the project is very low because it relies on free and open-source technologies such as React, JavaScript, HTML, and CSS.
Operating costs are minimal since the project uses Vercel for deployment and Firebase services for authentication and database management.
The project provides several benefits, including helping developers gain practical experience in building e-commerce websites and improving their frontend development skills.
The project also helps develop teamwork, problem-solving skills, and practical experience in web development.
Calculating the economic feasibility in terms of number:
Estimated monthly revenue: $2,500–$3,500 (based on sample order volume and advertising projections).
Estimated Annual Revenue:
Based on the estimated monthly revenue of $2,500–$3,500:
→ Annual revenue = $30,000–$42,000.

Estimated costs:

- Development Cost: 0 USD (using free and open-source technologies, Firebase free tier, and Vercel deployment)
- Hosting: 100–150 USD/year
- Maintenance: 100–200 USD/year

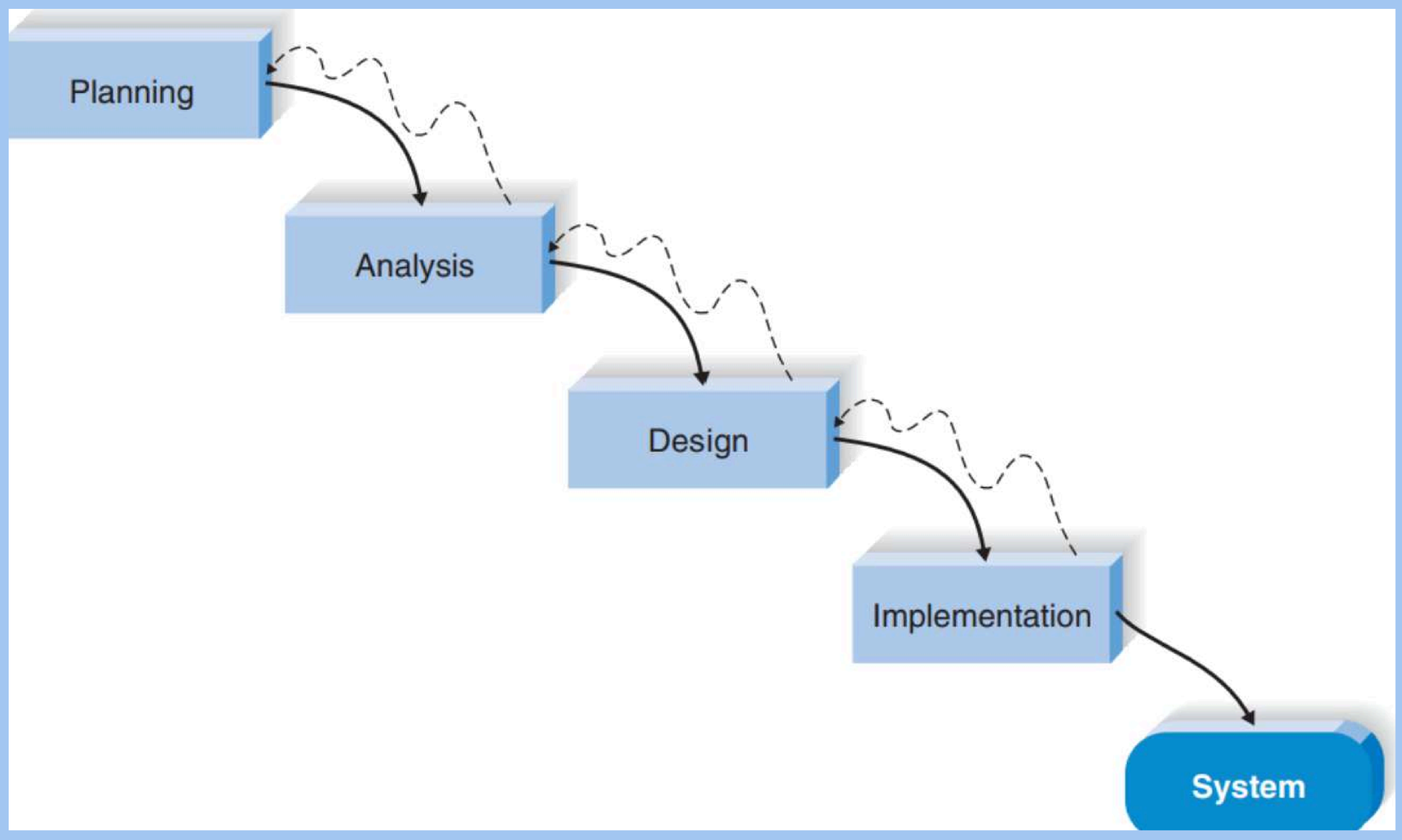
Total Estimated Cost (First Year): 200–350 USD.

Net Benefit:
$30,000–$42,000 – $200–$350
= approximately $29,650–$41,800 per year.

Organizational Feasibility :

The project is supported by the development team and aligns with the learning goals of the course.
Users will be able to browse products, add items to a shopping cart, place real orders, track orders, and view their order history through a cloud-based system.
Administrative users can manage products, users, and orders through a dedicated dashboard, while owner-level users have access to backup, restore, and product seeding features.
Other stakeholders, such as instructors and evaluators, are interested in the successful completion of the project.
The project supports the goal of learning modern frontend development and building practical web applications.

1.3 Project Methodology



Why We Chose the Waterfall Methodology ?

- **Clear and Well-Defined Requirements**
  - The project requirements are identified and defined from the very beginning.
  - This avoids confusion and reduces changes in the project.

- **Easy Planning and Scheduling**
  - The project methodology provides an easy way to divide project tasks into phases.
  - Each phase is provided with deadlines, which simplifies project management.

- **Sequential Development Process**
  - The project moves step by step from one stage to another.
  - This provides better control and organization in the project development process.

- **Suitable for Academic Projects**
  - The project scope is stable and does not undergo changes frequently.
  - The waterfall model is suitable for a structured environment like academic assignments.

- **Strong Documentation**
  - Each phase is fully documented (requirements, design, implementation, testing).
  - This provides better understanding, review, and maintenance of the project.

- **Better Progress Tracking**
  - The project progress can be tracked easily by completing phases step by step.
  - It becomes easier to track delays in the project.

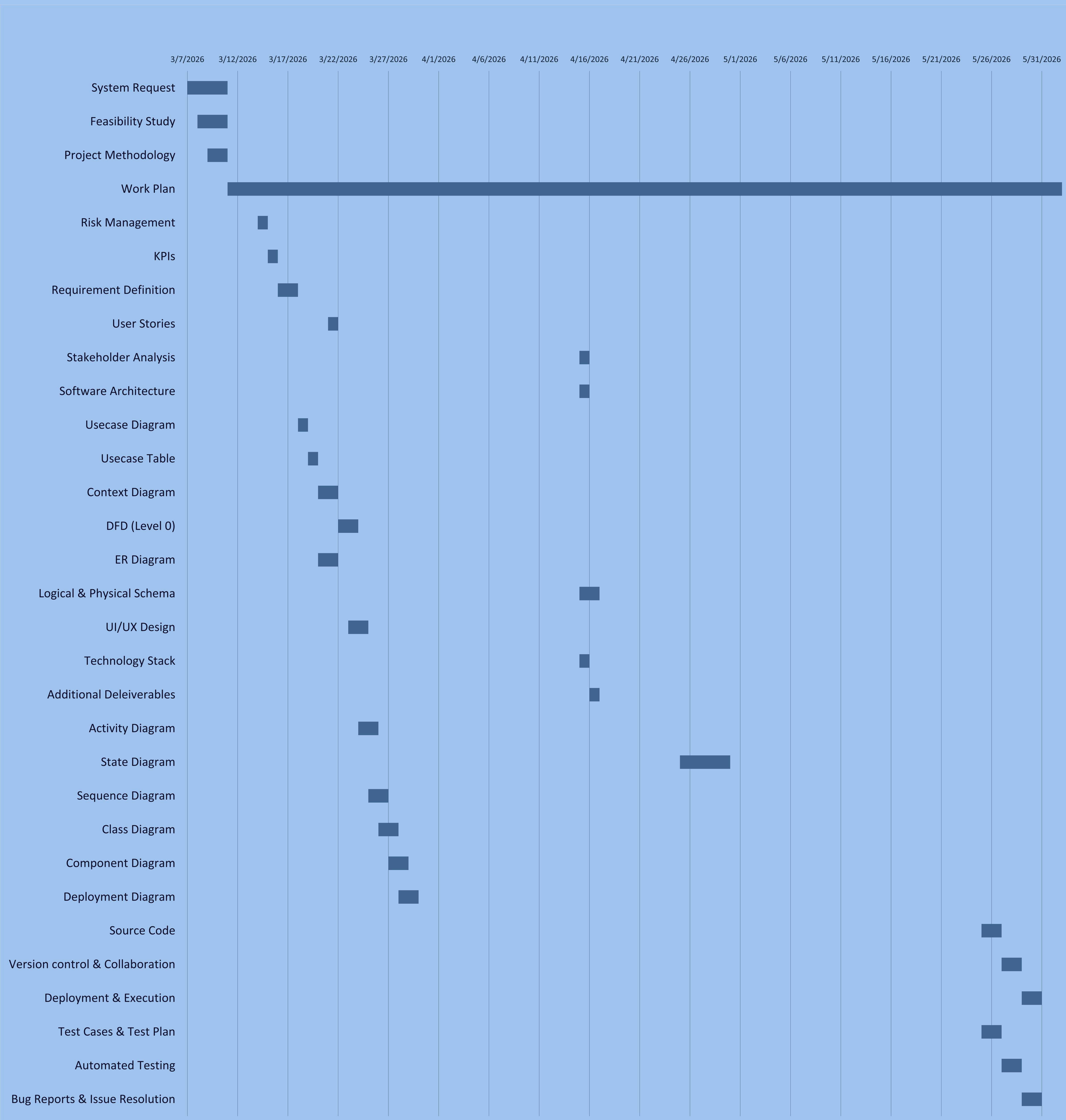
- **Clear Deliverables for Each Phase**
  - Each phase provides clear deliverables (diagrams, code, reports, etc.).
  - This provides better evaluation and quality in project deliverables.

1.4 Work Plan

1.4.1 Project Plan

Phase	Task ID	Task Name	Assigned To	Estimated			Actual			Dependency	Status
				Duration	Start Date	Finish Date	Duration	Start Date	Finish Date		
Planning	1.1	System Request	Hossam Hassan	4 Days	07 Mar 2026	11 Mar 2026	3 Days	07 Mar 2026	10 Mar 2026		✅ Completed
	1.2	Feasibility Analysis	Rawan Hamdi	3 Days	08 Mar 2026	11 Mar 2026	2 Days	08 Mar 2026	10 Mar 2026		✅ Completed
	1.3	Project Methodology	Mohamed Ahmed	2 Days	09 Mar 2026	11 Mar 2026	1 Day	09 Mar 2026	10 Mar 2026		✅ Completed
	1.4.1	Work Plan	Sohaib Ayman	127 Days	11 Mar 2026	16 Jul 2026	82 Days	11 Mar 2026	01 Jun 2026		✅ Completed
	1.5	Risk Management	Sohaib Ayman	1 Day	14 Mar 2026	15 Mar 2026	1 Day	14 Mar 2026	15 Mar 2026		✅ Completed
	1.6	KPIs	Sohaib Ayman	1 Day	15 Apr 2026	16 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
Analysis	2.1	Requirement Definition	Sohaib Ayman	2 Days	16 Mar 2026	18 Mar 2026	1 Day	18 Mar 2026	19 Mar 2026		✅ Completed
	2.2	User Stories	Sohaib Ayman	1 Day	21 Mar 2026	22 Mar 2026	1 Day	18 Mar 2026	19 Mar 2026		✅ Completed
	2.3	Stakeholder Analysis	Sohaib Ayman	1 Day	15 Apr 2026	16 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
	2.4	Software Architecture	Sohaib Ayman	1 Day	15 Apr 2026	16 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
	2.5.1	Usecase Diagram	Rawan Hamdi	1 Day	18 Mar 2026	19 Mar 2026	1 Day	18 Mar 2026	19 Mar 2026	2.1	✅ Completed
	2.5.2	Usecase Table	Rawan Hamdi	1 Day	19 Mar 2026	20 Mar 2026	1 Day	18 Mar 2026	19 Mar 2026	2.3.1	✅ Completed
	2.6	Context Diagram	Abdelrahman Khaled	2 Days	20 Mar 2026	22 Mar 2026	4 Days	18 Mar 2026	22 Mar 2026		✅ Completed
	2.7	DFD (Level 0)	Rawan Hamdi	2 Days	22 Mar 2026	24 Apr 2026	1 Day	26 Apr 2026	27 Apr 2026		✅ Completed
	2.8	ER Diagram	Mohamed Ahmed	2 Days	20 Mar 2026	22 Mar 2026	3 Days	18 Mar 2026	21 Mar 2026		✅ Completed
	2.9	Logical & Physical Schema	Sohaib Ayman	2 Days	15 Apr 2026	17 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
Design	3.1	UI/UX Design	Sohaib Ayman	2 Days	23 Mar 2026	25 Mar 2026	1 Day	10 Apr 2026	11 Apr 2026		✅ Completed
	3.2	Technology Stack	Sohaib Ayman	1 Day	15 Apr 2026	16 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
	3.3	Additional Deliverables	Sohaib Ayman	1 Day	16 Apr 2026	17 Apr 2026	1 Day	15 Apr 2026	16 Apr 2026		✅ Completed
	3.4.1	Activity Diagram	Sohaib Ayman	2 Days	24 Mar 2026	26 Mar 2026	1 Day	11 Apr 2026	12 Apr 2026	2.5.1	✅ Completed
	3.4.2	State Diagram	Mohamed Ahmed	5 Days	25 Apr 2026	30 Apr 2026	13 Days	25 Apr 2026	08 May 2026		✅ Completed
	3.4.3	Sequence Diagram	Rawan Hamdi	2 Days	25 Mar 2026	27 Mar 2026	2 Days	10 Apr 2026	12 Apr 2026		✅ Completed
	3.4.4	Class Diagram	Hossam Hassan	2 Days	26 Mar 2026	28 Mar 2026	1 Day	10 Apr 2026	11 Apr 2026	2.7	✅ Completed
	3.4.5	Component diagram	Hossam Hassan	2 Days	27 Mar 2026	29 Mar 2026	1 Day	11 Apr 2026	12 Apr 2026		✅ Completed
	3.4.6	Deployment Diagram	Mohamed Ahmed	2 Days	28 Mar 2026	30 Mar 2026	1 Day	12 Apr 2026	13 Apr 2026		✅ Completed
Implementation	4.1	Source code	Hossam Hassan	2 Days	25 May 2026	27 May 2026	2 Days	25 May 2026	27 May 2026		✅ Completed
	4.2	Version Control & Collaboration	Hossam Hassan	2 Days	27 May 2026	29 May 2026	2 Days	27 May 2026	29 May 2026		✅ Completed
	4.3	Deployment & Execution	Sohaib Ayman	2 Days	29 May 2026	01 Jun 2026	2 Days	29 May 2026	01 Jun 2026		✅ Completed
Testing	5.1	Test Cases & Test Plan	Rawan Hamdi	2 Days	25 May 2026	27 May 2026	2 Days	25 May 2026	27 May 2026		✅ Completed
	5.2	Automated Testing	Abdelrahman Khaled	2 Days	27 May 2026	29 May 2026	2 Days	27 May 2026	29 May 2026		✅ Completed
	5.3	Bug Reports & Issue Resolution	Sohaib Ayman	2 Days	29 May 2026	01 Jun 2026	2 Days	29 May 2026	01 Jun 2026		✅ Completed

1.4.2 Gantt Chart



1.5 Risk Management

Risk management is an essential part of the project to identify potential issues that may affect development, performance, or delivery. These risks are analyzed and handled using appropriate mitigation strategies to ensure the success of the system.

Risk	Description	Mitigation	Priority
Time Constraints	Limited time to complete project tasks.	Develop a project timeline and prioritize key features.	High
Technical Issues	Issues that may be experienced in the implementation of the code or integration of the API.	Regularly test the project and use the right tools and resources.	High
Team Coordination	Delays or improper distribution of tasks to the team members.	Properly distribute tasks and maintain communication with the team.	Medium
Requirement Changes	Changes in the project requirements during the development stage.	Develop the project requirements early and avoid unnecessary changes.	Medium
Data Loss	Loss of the project code or data.	Use GitHub as the version control tool.	High
Performance Issues	Poor performance of the system or delayed response when loading data.	Optimize the code and reduce the number of API requests.	Medium

Risk Monitoring :

All risks identified will be monitored during the entire project life cycle. The team will regularly monitor their progress and also be able to identify new risks that may occur.

1.6 KPIs (Key Performance Indicators)

1. Response Time
Determines how fast the system reacts to the user when performing certain actions, such as loading the products and adding them to the cart.

- **Objective: less than 2 seconds**

2. System Availability (Uptime)
Shows how many times the system is available and works correctly.

- **Objective: More than 95%**

3. User Adoption Rate
The indicator that shows how many registered users are actively using the app.

- **Objective: The number of active users is increasing**

4. Cart Completion Rate
Indicates the percentage of users who proceed from adding products to the cart to successfully placing an order.

- **Objective: More than 60%**

5. Error Rate
Measures how many times an error happens (for instance, broken API or action).

- **Objective: Less than 5%**

6. Page Load Time
The time needed for loading the pages: Home, Products.

- **Objective: Less than 3 seconds**

7. Database Operation Success Rate
Measures how often Firestore operations such as reading products, retrieving orders, updating order status, and managing users are completed successfully.

- **Objective: 95% successful operations.**

8. Data Persistence Accuracy

- **The measure of correctness of storing and retrieving information from Redux state, LocalStorage, and Cloud Firestore.Objective: 100%**

9. Order Processing Accuracy
Measures the percentage of orders that are successfully created, stored, retrieved, and tracked without data inconsistencies.

- **Objective: 100%**

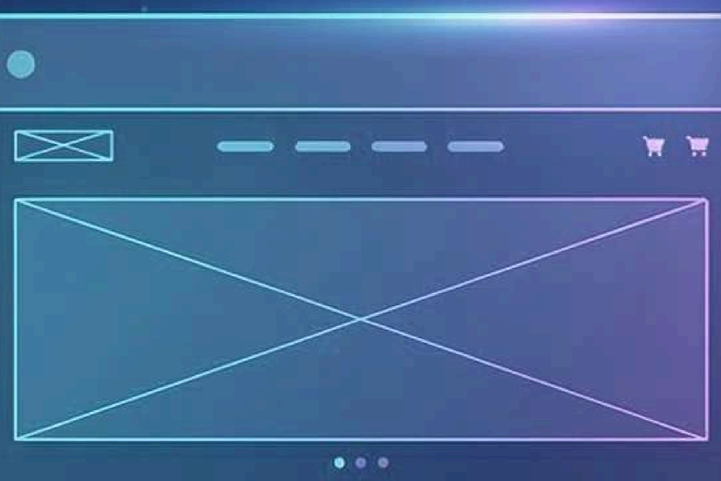
ANALYSIS

SALES ANALYTICS CHARTS

REVENUE GROWTH

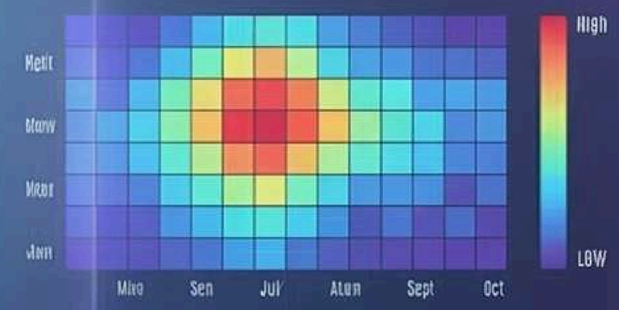
PRODUCT SALES

TRAFFIC STATISTICS



CUSTOMER BEHAVIOR ANALYSIS

VISITOR ACTIVITY



CONVERSION RATES

PRODUCT PERFORMANCE INSIGHTS

BEST SELLING ITEMS

TRENDS

- Revenue tech analysis product dashboard and performers
- Trends analysis

2.1 Requirement Definition

Functional Requirements:

1. Customers
   1.1 The customer can sign up and create an account.
   1.2 The customer can log in using a username and password.
   1.3 The customer can browse products without creating an account using guest access.
   1.4 The customer can view the products by the defined categories.
   1.5 The customer can search the products within the selected category.
   1.6 The customer can get the details of the product.
   1.7 The customer can add the products to the cart.
   1.8 The customer can change the quantity of the products.
   1.9 The customer can remove the products from the cart.
   1.10 The customer can add the account.
   1.11 The customer can proceed to the checkout.
   1.12 The customer can see the order summary.
   1.13 The customer can complete the purchase.
   1.14 The customer can track orders using an order ID.
   1.15 The customer can view order history after logging in.
   1.16 The customer can switch between light mode and dark mode.

2. Admin
   2.1 The admin can add new products into the system.
   2.2 The admin can edit the information of existing products.
   2.3 The admin can remove products from the system.
   2.4 The admin can categorize the products.
   2.5 The admin can view the registered users.
   2.6 The admin can remove users from the system.
   2.7 The admin can view customer orders.
   2.8 The admin can update order status.

3. Owner
   3.1 The owner can create product backups.
   3.2 The owner can restore products from backups.
   3.3 The owner can seed products from the Platzi API.
   3.4 The owner can access all administrative functions.

4. System
   4.1 The system updates the cart dynamically without reloading the page.
   4.2 The system displays the cart preview when the cart icon is hovered.
   4.3 The system displays the number of items in the cart.
   4.4 The system stores cart data using localStorage and stores application data using Cloud Firestore.
   4.5 The system manages the routing between pages.
   4.6 The system synchronizes products, users, and orders with Cloud Firestore.
   4.7 The system authenticates users using Firebase Authentication.
   4.8 The system supports role-based access control for User, Admin, and Owner roles.

Nonfunctional Requirements:

1. Operational
   1.1 The application must be able to run smoothly on various supported devices and browsers.
   1.2 The application must be available most of the time.
   1.3 The application must preserve Firestore data integrity during updates and deployments.

2. Performance
   2.1 The application should take a few seconds to load.
   2.2 The application should respond quickly when the user interacts with it.
   2.3 The application should retrieve and synchronize Firestore data efficiently.

3. Security
   3.1 The application should validate user data.
   3.2 Only logged-in users should be able to proceed to the checkout.
   3.3 User authentication and database access should be secured using Firebase Authentication and Firestore Security Rules.

4. Cultural and political
   4.1 No special cultural and political requirements are expected.

2.2 User Stories

Title	User Story	Acceptance Criteria	Priority
User Login	As a user, I want to log in with my username and password.	• The system validates the username. • The password is at least 6 characters. • An error message is shown if the login is not successful. • The user is redirected if the login is successful.	High
Browse Products by Category	As a customer, I want to browse products by category.	• Products are shown according to the category selected. • Each product has an image, name, and price. • Navigation between categories is correct.	High
Search and Filter Products	As a customer, I want to search products.	• The user is able to search products within the selected category. • Products are filtered according to the price. • Products are sorted according to the price, from low to high, and vice versa.	Medium
Add Product to Cart	As a customer, I want to add products to the cart so that I can purchase multiple items.	• The user can add a product to the cart. • The cart updates without page reload. • The cart counter updates correctly. • The product appears in the cart preview.	High
Checkout Process	As a customer, I want to complete the checkout process so that I can place my order.	• Only logged-in users can proceed to checkout. • If not logged in, the user is redirected to the login page. • The system displays an order summary. • The user can confirm the order successfully.	High
Manage Cart	As a customer, I want to update and remove items from my cart so that I can manage my cart before checking out.	• User is able to increase or decrease product quantity. • User is able to remove products from their cart. • User sees a message before removing an item from their cart. • User sees an automatic change in the price after updating their cart.	High

2.3 Stakeholder Analysis

Definition
In stakeholder analysis, we find out who the key players are in the system and identify their requirements.

1. Customer (end-users)
These people use the Marketly system directly.
Requirements:

- **Convenient search and navigation**
- **Easy adding products into cart and buying them**
- **Order tracking**
- **Order history**
- **Guest browsing support**
- **Dark mode support**
- **Secure authentication (login/register through Firebase)**

2. Administrator
Administrator manages the system.
Requirements:

- **Ability to add, modify and delete items**
- **Managing categories**
- **Tracking order status**
- **Managing users**
- **Having access to a convenient dashboard**

3. System Owner
System owner is the person that owns the Marketly system.
Requirements:

- **A high level of performance**
- **A good user experience**
- **Few errors**
- **Scalability potential for the future**
- **Database backup and recovery**
- **Data protection**
- **System maintenance capabilities**

4. Third-party services
Firebase Authentication
Third-party service for user login.
Requirements:

- **Integration with the frontend**
- **Handling credentials safely**
- **Secure authentication process**

Cloud Firestore
Primary database service used by the application.
Requirements:

- **Real-time data synchronization**
- **Secure data storage**
- **Reliable read and write operations**
- **Scalable cloud infrastructure**

Platzi API
External service used for product seeding operations.
Requirements:

- **Reliable product retrieval**
- **Stable API connectivity**
- **Accurate product data import**

5. Developers
People responsible for creating and managing the system.
Requirements:

- **Clean code**
- **Nice architecture**
- **Debugging**
- **Reusable parts**
- **Maintainable codebase**
- **Firestore integration**
- **Performance optimization**
- **Role-based access control implementation**

2.4 Software Architecture

1. Introduction
Marketly uses a client-side architecture with integrated cloud services.
React is used as the frontend framework, while Firebase Authentication and Cloud Firestore provide authentication and database functionality. Platzi API is used only for product seeding operations.

2. Architecture Pattern Used
The application follows a component-based architecture approach but with some client-server architecture features.

- **Frontend is made up of reusable components**
- **APIs are used to communicate with the external services**
- **There is no dedicated backend server. Firebase services provide authentication and database functionality through a Backend-as-a-Service (BaaS) architecture.**

3. System layers

1. **Presentation Layer (Frontend UI)**
    - **React**
    - **Purpose:**
        - **Display UI**
        - **User Interactions**
        - **Navigation between pages**
2. **Application Layer (Frontend Logic)**
    - **Made up of business logic in React**
    - **Includes:**
        - **Product Service**
        - **Cart Service**
        - **Order Logic**
        - **Authentication Logic**
        - **User Management Logic**
        - **Order Tracking Logic**
        - **Admin Dashboard Logic**
        - **Backup & Restore Logic**
3. **External Services Layer**
    - **Firebase Authentication**
        - **Used for login and sign-up functionalities**
        - **Manages secure sessions of users**
    - **Cloud Firestore**
        - **Stores products, categories, users, orders, and backup data.**
        - **Provides real-time database synchronization.**
    - **Platzi API**
        - **Used for product seeding operations.**
        - **Imports products into Cloud Firestore.**

4. Component Interactions

- **React app interacts with:**
  - **Firebase Authentication for user authentication**
  - **Cloud Firestore for products, categories, users, orders, and backups**
  - **Platzi API for product seeding operations**
- **Components handle:**
  - **Cart functions**
  - **Ordering**
  - **Data management**

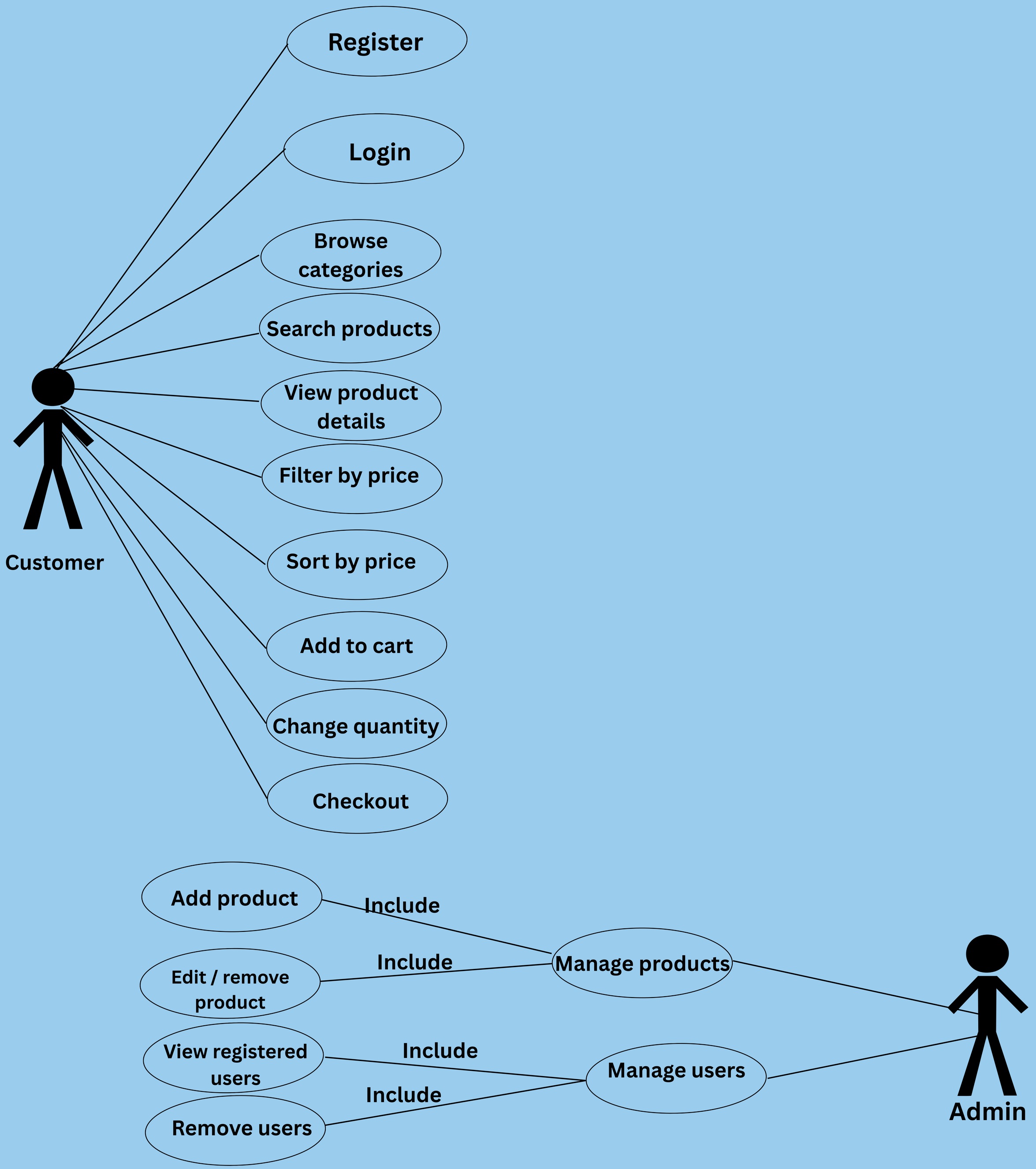
5. Data flow

1. **The user interacts with the React front-end UI**
2. **Requests are processed by the app**
3. **Where necessary:**
   - **Products and categories are retrieved from Cloud Firestore**
   - **Authentication is performed using Firebase Authentication**
   - **Orders are stored and retrieved from Cloud Firestore**
   - **Cart data is stored in LocalStorage**
4. **Product seeding data is imported from Platzi API into Cloud Firestore**

6. Advantages of the Architecture

- **Simple and lightweight design (no need for a backend server)**
- **Ease of development and maintenance**
- **Scalable to support future backends**
- **Use of trustworthy external services**
- **Real-time database synchronization**
- **Cloud-based data persistence**
- **Role-based access control**
- **Scalable database architecture using Firestore**

2.5.1 Usecase Diagram

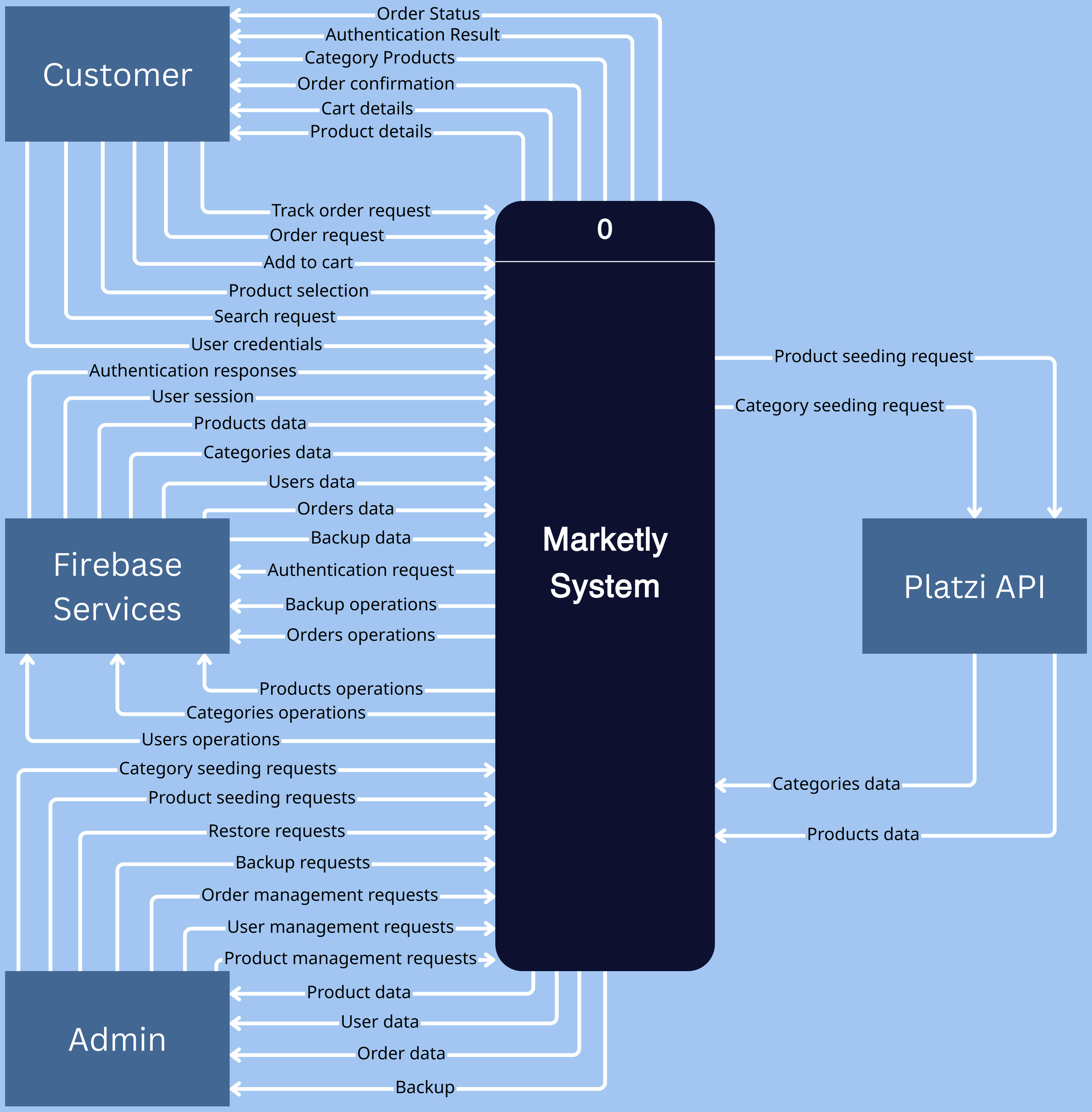


2.5.2 Usecase Table

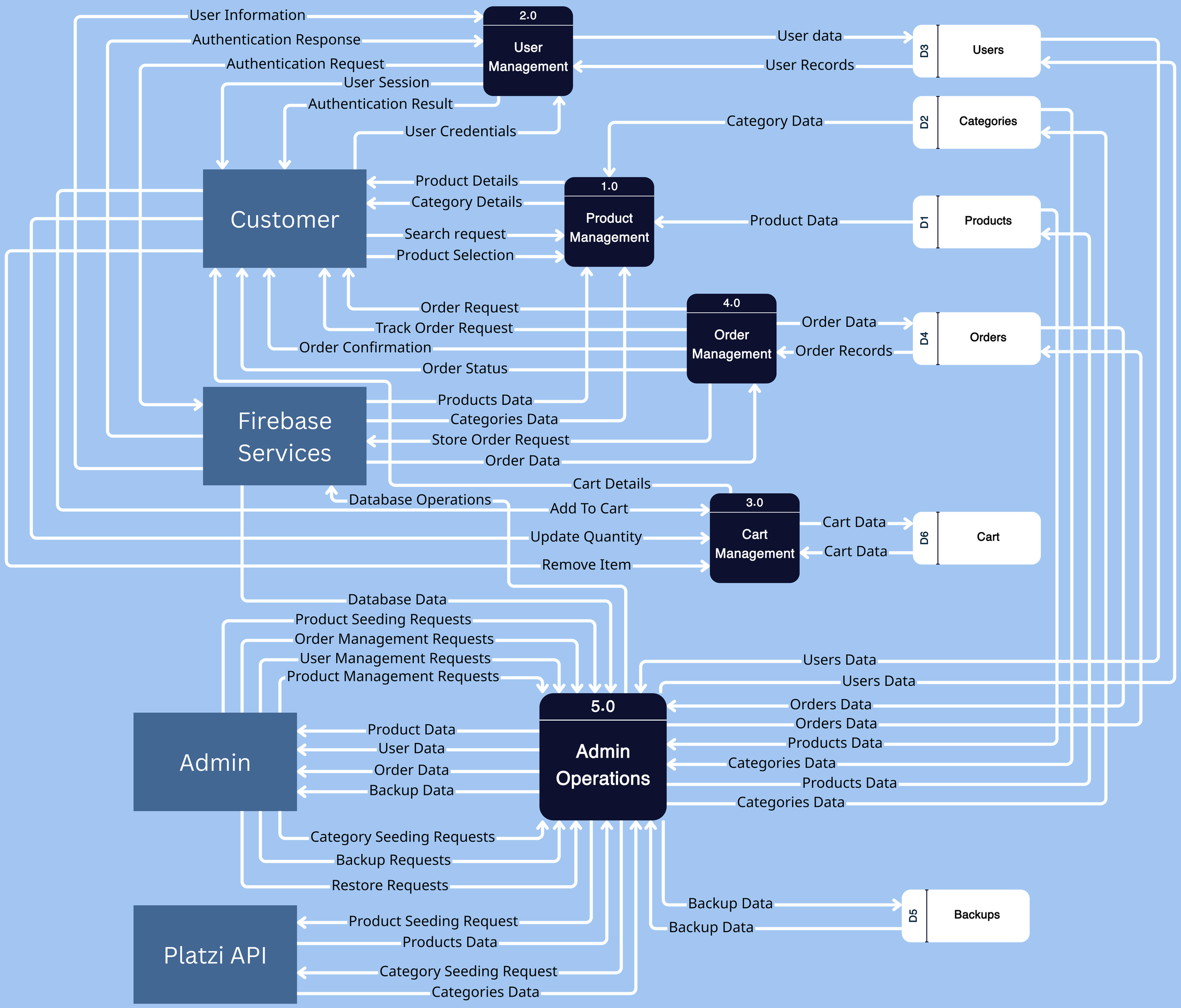
Marketly — Use Case Table

ID	USE CASE NAME	ACTOR	DESCRIPTION	PRECONDITION	MAIN FLOW	POSTCONDITION	PRIORITY
UC-1	Register	Customer	New user creates an account on Marketly.	User has no existing account.	User fills registration form → System validates → Account created → Redirect to home.	Account is created and user is logged in.	Medium
UC-2	Login	Customer	Registered user logs into their account.	User has a registered account.	User enters email and password → System validates → Session started → Redirect to home.	User is authenticated and redirected to home page.	High
UC-3	Browse Categories	Customer	User browses product categories to find items.	User is on the home page.	User clicks a category → System retrieves products → Product list displayed.	Products of the selected category are displayed.	High
UC-4	Search Products	Customer	User searches for products using a keyword.	User is on any page with a search bar.	User types keyword → System filters products → Results displayed.	Matching products are displayed.	High
UC-5	View Product Details	Customer	User views full details of a selected product.	User is browsing a product list.	User clicks a product → System retrieves details → Product details page displayed.	Product details page is shown to the user.	High
UC-6	Filter by Price	Customer	User filters products by a price range.	User is viewing a product list.	User sets min/max price → System filters → Filtered products displayed.	Only products within the price range are shown.	Medium
UC-7	Sort by Price	Customer	User sorts products by price order.	User is viewing a product list.	User selects sort option → System re-orders list → Sorted products displayed.	Products are displayed in the selected sort order.	Low
UC-8	Add to Cart	Customer	User adds a product to the shopping cart.	User is viewing a product details page.	User clicks Add to Cart → System adds product → Cart counter updates → Saved to LocalStorage.	Product is added to cart and persisted in LocalStorage.	High
UC-9	Remove from Cart	Customer	User removes a product from the cart.	User has at least one item in the cart.	User clicks remove → System removes product → Cart updated → LocalStorage updated.	Product is removed and cart is updated.	Medium
UC-10	Change Quantity	Customer	User changes the quantity of a cart item.	User has at least one item in the cart.	User clicks + or - → System updates quantity → Total recalculated → LocalStorage updated.	Cart quantity and total are updated and persisted.	Medium
UC-11	Checkout	Customer	Logged-in user completes a purchase.	User is logged in and has items in cart.	User clicks Checkout → System shows order summary → User confirms → Order processed → Cart cleared.	Order is placed and cart is cleared from LocalStorage.	High
UC-12	Manage Products	Admin	Admin adds, edits, or removes products from the system.	Admin is logged in with management privileges.	Admin opens panel → Selects action (add/edit/remove) → System performs action → List updated.	Product list is updated in the system.	High
UC-13	Manage Users	Admin	Admin views and removes registered users.	Admin is logged into the system.	Admin opens user panel → Views user list → Selects user → System removes user → List updated.	User is removed from the system.	Medium

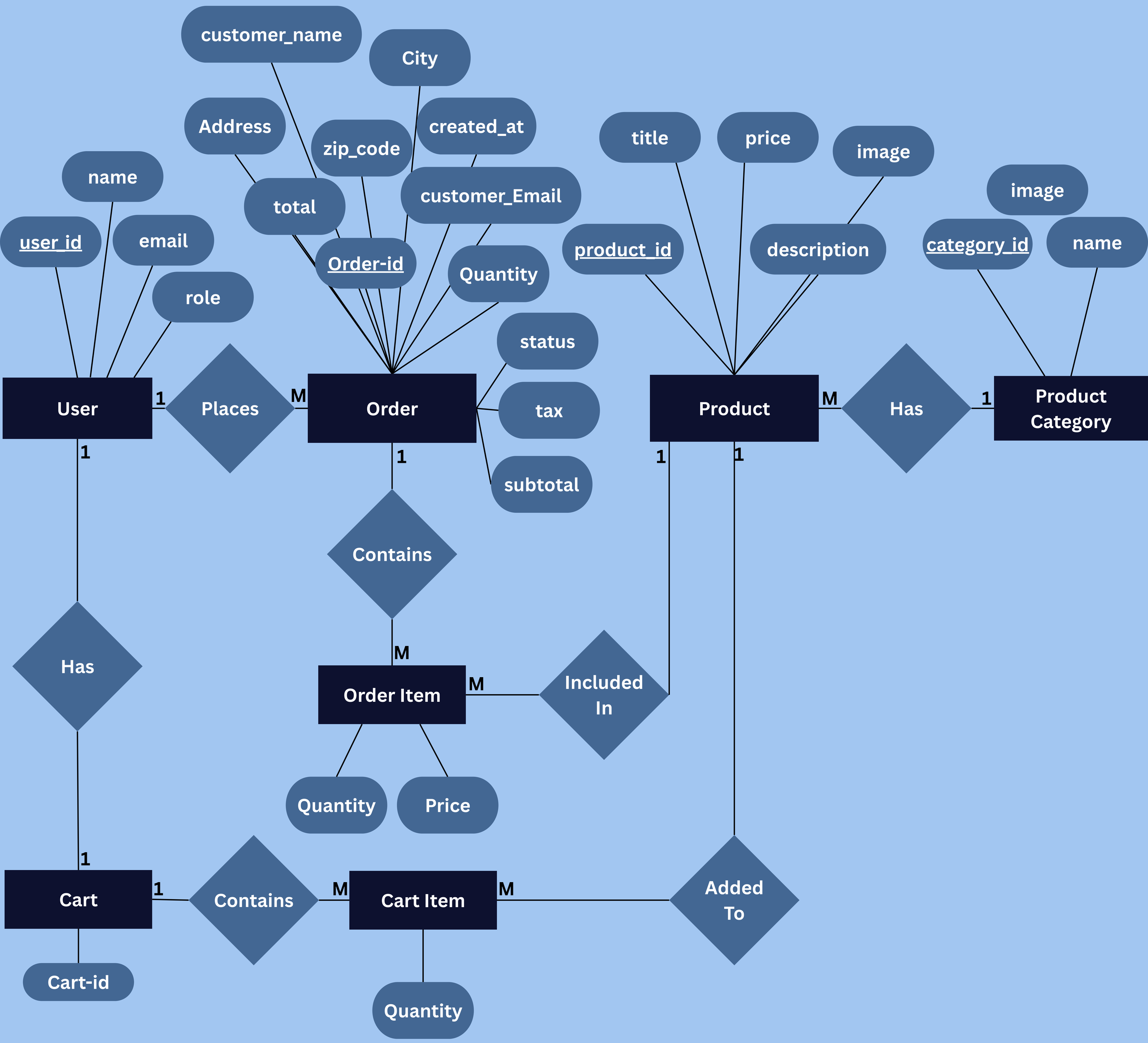
2.6 DFD (Level 0)



2.7 DFD (Level 0)



2.8 ER Diagram



2.9 Logical & Physical Schema

2.9.1 Logical Schema

The system consists of the following main entities:

User

user_id	email

Product

product_id	title	slug	price	category	description	thumbnail	images	creationAt

Cart

cart_id	user_id

Cart_Item

cart_item_id	cart_id	product_id	quantity

Order

order_id	user_id	subtotal	tax	total	status	createdAt
customer_name	customer_email	customer_address				
customer_city	customer_zip	customer_name				

Order_Item

order_item_id	order_id	product_id	quantity	price

Category

category_id	name	slug	image

Backup

backup_id	type	createdAt

2.9.2 Physical Schema

Introduction

The system does not make use of any traditional relational database.

In its place, it uses:

- **Firebase Authentication for user login information**
- **LocalStorage for storing data of the application**
- **Cloud Firestore for storing application data**
- **Platzi API for product seeding operations**

1. User Information (Firebase Authentication)

The credentials of users are stored externally via Firebase.

Storing Data:

- **user_id (UID)**
- **email**
- **password (encrypted)**

2. Structure of LocalStorage

- **Cart Data**

```
{
  "cart": [
    {
      "product_id": 1,
      "quantity": 2
    }
  ]
}
```

- **Theme Preferences**

```
{
  "theme": "dark"
}
```

3. Cloud Firestore Collections

- **Products Collection**

```
{
 "id": 1,
 "title": "Product Name",
 "slug": "product-name",
 "category": "clothes",
 "description": "Product description",
 "price": 100,
 "images": ["image_url"],
 "thumbnail": "image_url",
 "creationAt": "timestamp"
}
```

- **Categories Collection**

```
{
 "id": 1,
 "name": "Clothes",
 "slug": "clothes",
 "image": "image_url"
}
```

- **Users Collection**

```
{
 "uid": "user_uid",
 "email": "user@email.com",
 "role": "user"
}
```

- Orders Collection


```
{
 "orderId": "ORD-2026-269",
 "userId": "user_uid",
 "status": "Processing",
 "subtotal": 100,
 "tax": 10,
 "total": 110,
 "createdAt": "timestamp",
 "customer": {
 "name": "Customer Name",
 "email": "customer@email.com"
 },
 "items": [
 {
 "id": 1,
 "title": "Product Name",
 "quantity": 2,
 "price": 50
 }
 ]
}
```


- Backups Collection
  - Stores backup copies of products and categories used by the restore system.

4. Storing Data Strategy

- Cart data is stored in LocalStorage.
- Products, categories, users, orders, and backups are stored in Cloud Firestore.
- User authentication is carried out externally through Firebase
- Product data is retrieved from an external API

5. Data Consistency Note

Cloud Firestore acts as the central source of truth for application data, ensuring consistency and synchronization across all connected clients.

DESIGN

HOMEPAGE LAYOUT WIREFRAME

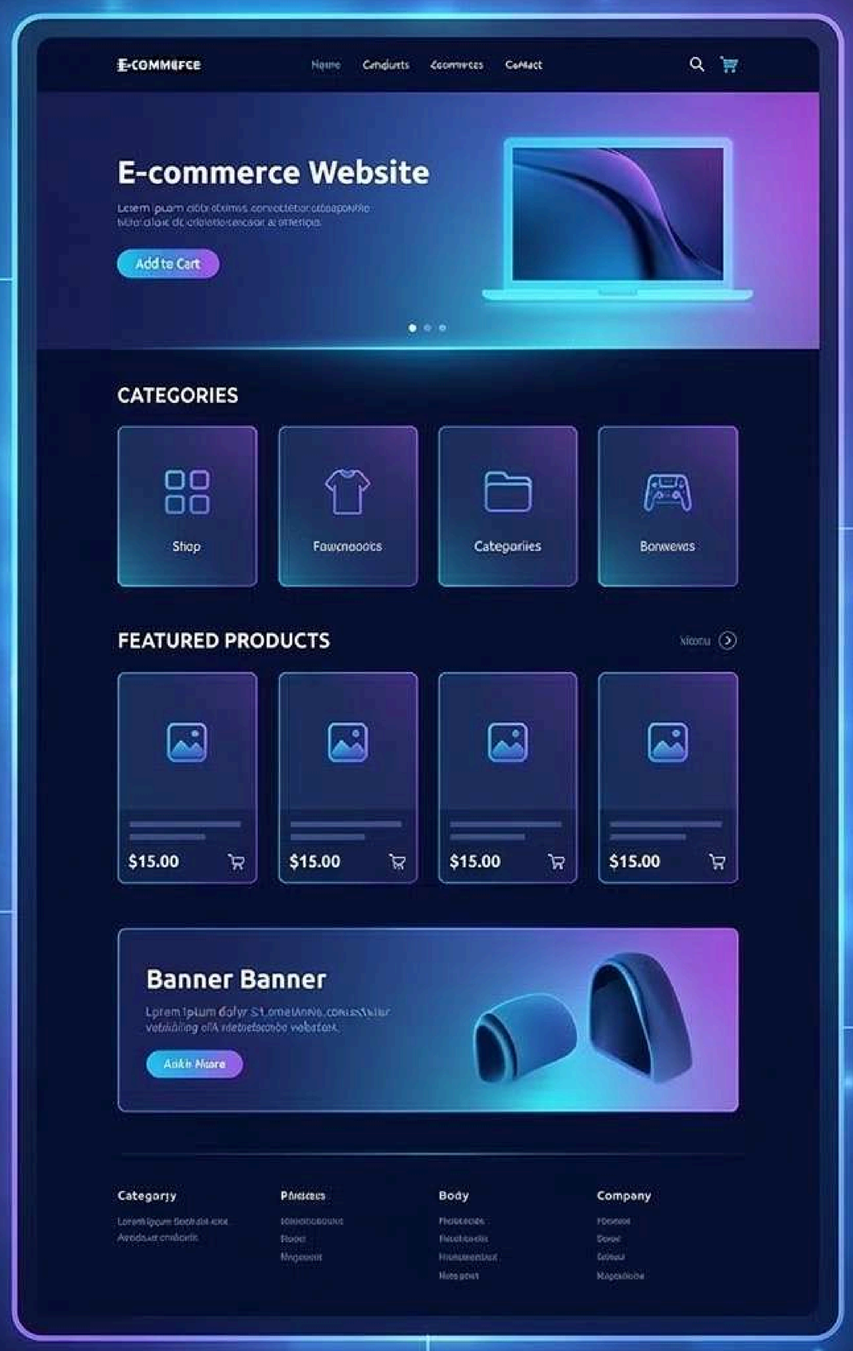
UI COMPONENTS

BUTTONS

Add to Cart | Add to Cart | Learn More | Learn More

PRODUCT CARDS

NAVIGATION BAR



COLOR PALETTE AND TYPOGRAPHY PANEL

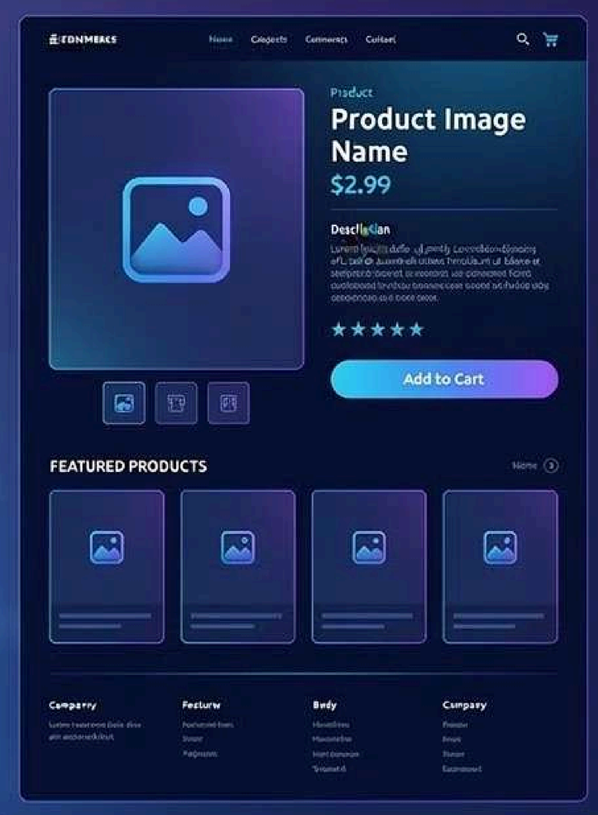
FONT: **[Modern Sans]**

SIZES: **Heading**

Body **Heading**

Small **Body**

PRODUCT PAGE DESIGN PREVIEW



RESPONSIVE DESIGN PREVIEWS

RESPONSIVE DESIGN PREVIEWS

MOBILE

DESKTOP

3.1 UI/UX Design

Users: Anyone who want to browse, purchase products, and manage their orders online.

Functions:

- A user can register and log in to the system.
- A user can browse products by categories.
- A user can search for products.
- A user can view product details.
- A user can add products to the cart.
- A user can update or remove items from the cart.
- A user can place orders.
- A user can track their orders.
- A user can view order history.
- An admin can manage products (add, edit, delete).
- An admin can categorize products.
- An admin can view users and manage them.

Step 1: User Scenario Development

Process: 1

Login or Create Account

1.	**As a Guest:** 1.1 User opens the website. 1.2 User browses products without logging in.
2.	**Login:** 2.1 If the user has an account. 2.2 User enters email and password. 2.3 User is redirected to the homepage.
3.	**Register:** 3.1 If the user does not have an account. 3.2 User enters: • Username • Email • Password 3.3 User clicks “Register”. 3.4 Account is created successfully.

Process: 2

Browse & Add to Cart

1.	1.1 User opens homepage. 1.2 User selects a category.
2.	2.1 System displays products. 2.2 User clicks on a product.
3.	3.1 User views product details. 3.2 User clicks “Add to Cart”.
4	4.1 Product is added to cart. 4.2 Cart is updated.

Process: 3

Place Order

1.	1.1 User opens cart.
2.	2.1 User reviews items. 2.2 User clicks “Checkout”.
3.	3.1 If not logged in → system asks to login. 3.2 if logged in → continue.
4	4.1 Order is places. 4.2 Confirmation message appears.

Process: 4

Track Order

1.	1.1 User goes to “My Orders”. 1.2 User selects an order.
2.	2.1 System displays order status. 2.2 User tracks progress (Processing/Shipped/Delivered).

Process: 5

Admin Manage Products

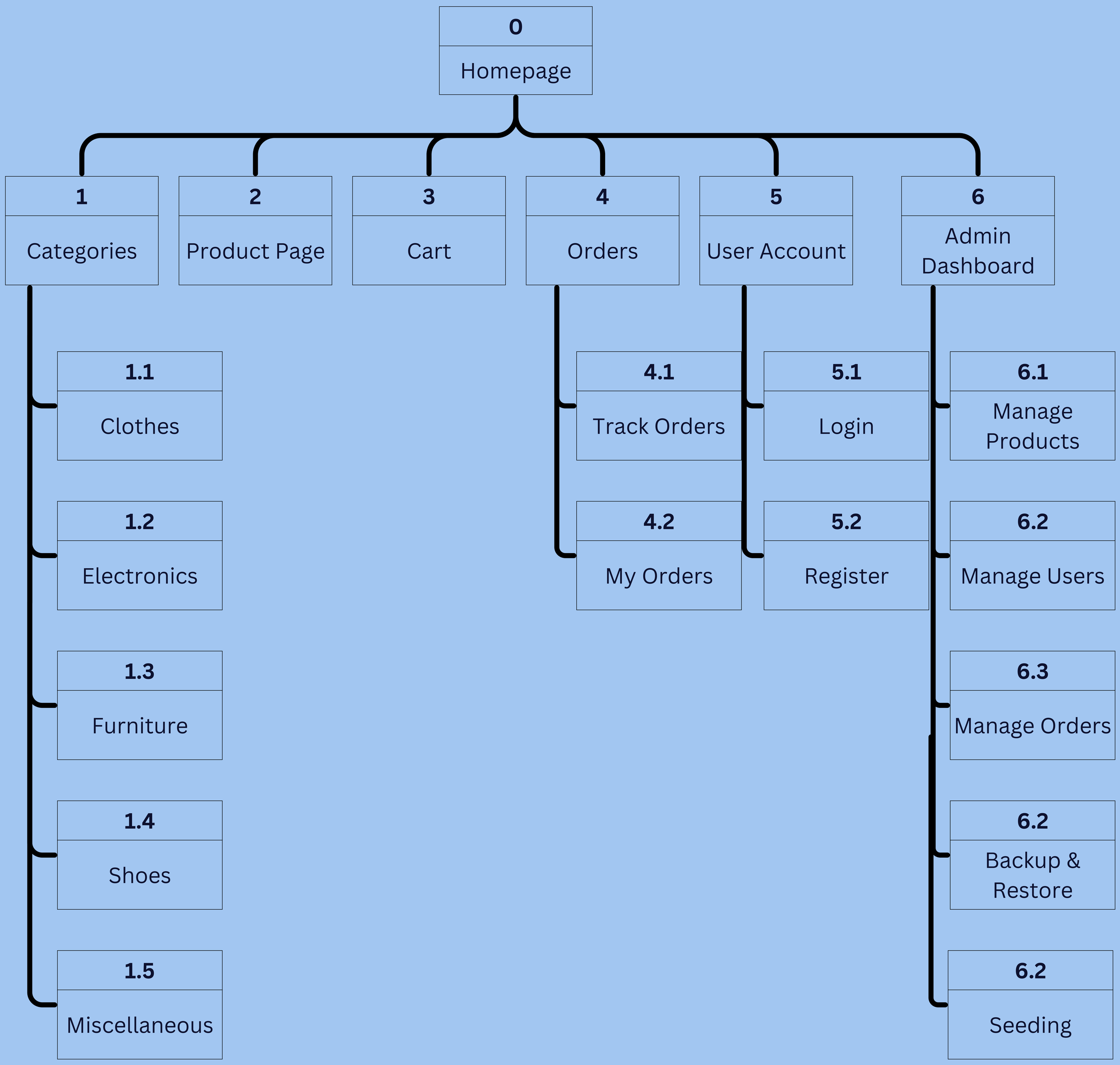
1.	1.1 Admin login into the website. 1.2 Admin opens the dashboard.
2.	2.1 Website shows list of products. 2.2 Admin chooses to add / edit / delete.
3.	**Adding a New Product:** 3.1.1 Admin clicks Add Product. 3.1.2 Admin fills details of the product (product name, price, category, description, picture). 3.1.3 Admin submits the form. 3.1.4 Product is added.
4.	**Editing Product Details:** 4.1.1 Admin selects the product from the list. 4.1.2 Admin edits product details. 4.1.3 Admin saves the details. 4.1.4 Product is edited.
5.	**Deleting Product Details:** 5.1.1 Admin selects the product. 5.1.2 Admin clicks delete option. 5.1.3 Website prompts for confirmation. 5.1.4 Admin confirms. 5.1.5 Product gets deleted.

Process: 6

Admin Manage Orders

1.	1.1 The admin signs in to the website. 1.2 The admin goes to the Dashboard page.
2.	2.1 The admin clicks on the "Orders" tab. 2.2 The system shows the order list.
3.	3.1 The admin clicks on any order. 3.2 The system shows the order details.
4.	4.1 The admin chooses order status from Processing/ Shipped/Delivered. 4.2 The admin changes order status.
5.	5.1 The system updates order status. 5.2 Order status is changed successfully.
6.	6.1 User sees updated order status in "My Orders".

Step 2: Interface Structure Diagram (ISD)



Step 3: Interface Standard Design

Interface Metaphor

Online shopping store (digital marketplace).

2. Interface Objects

- Product Card: display image, name, and price
- Cart: stores selected items
- Order Trackers: shows order status
- Search Bar: finds products
- Category Cards: organizes products

3. Interface Actions

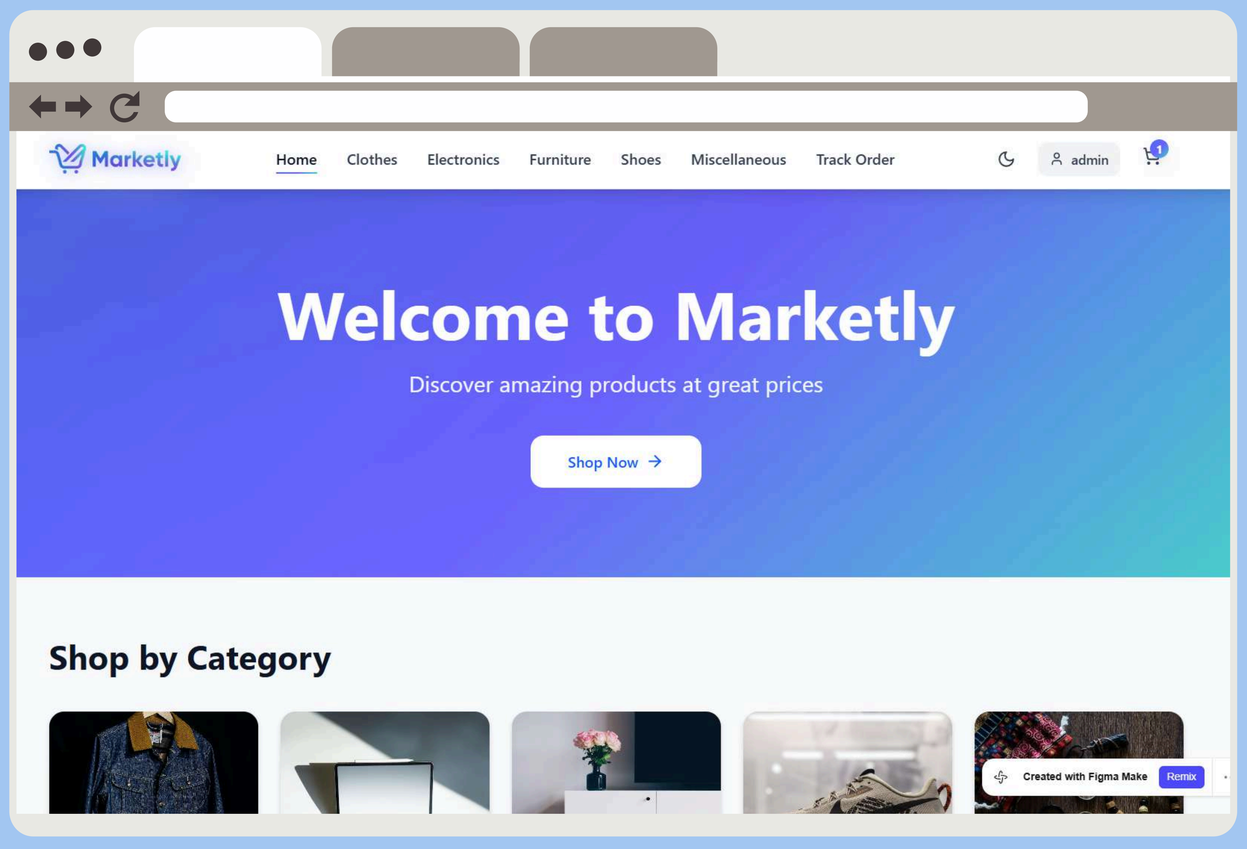
- Add to Cart
- Remove from Cart
- Checkout
- Track Order
- Manage Products (Admin)
- Manage Users (Admin)
- Manage Orders (Admin)
- Backup (Owner)
- Restore (Owner)

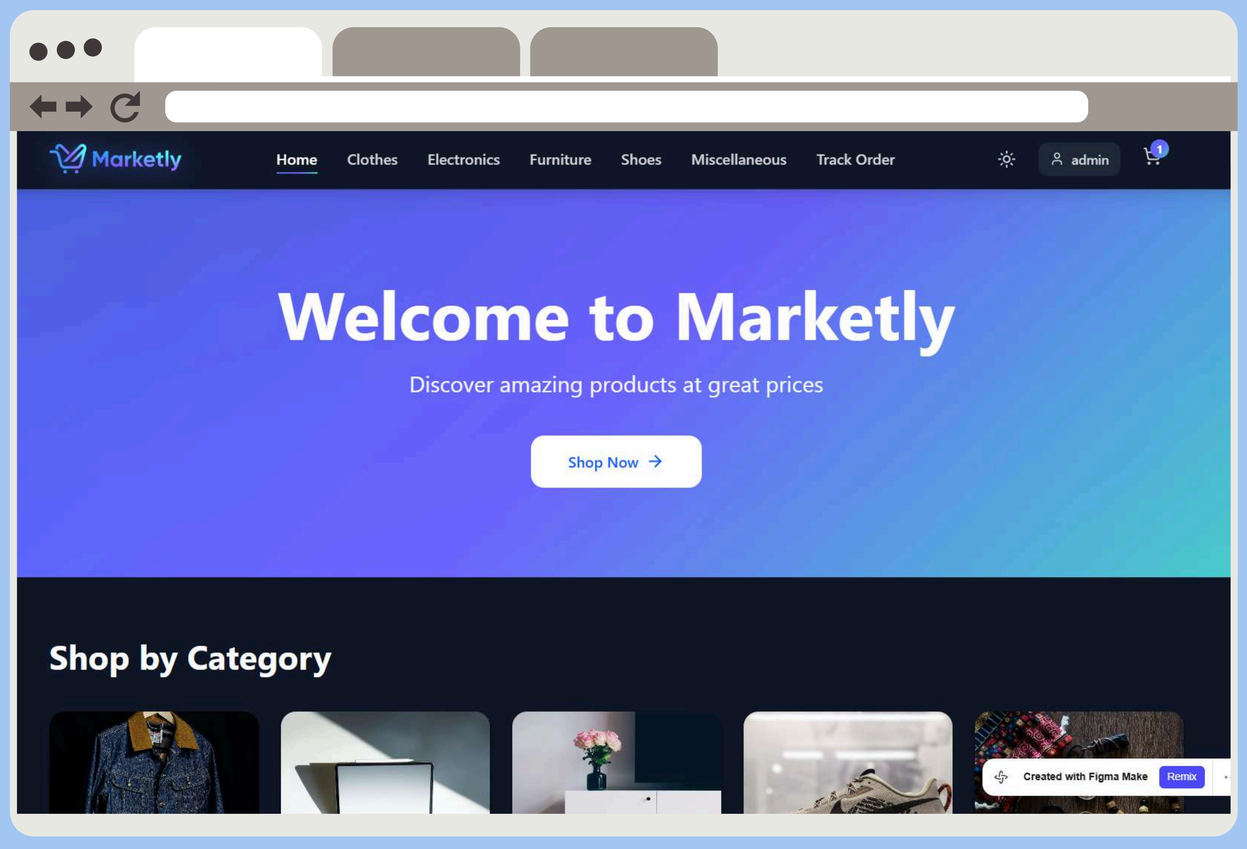
4. Interface Icons

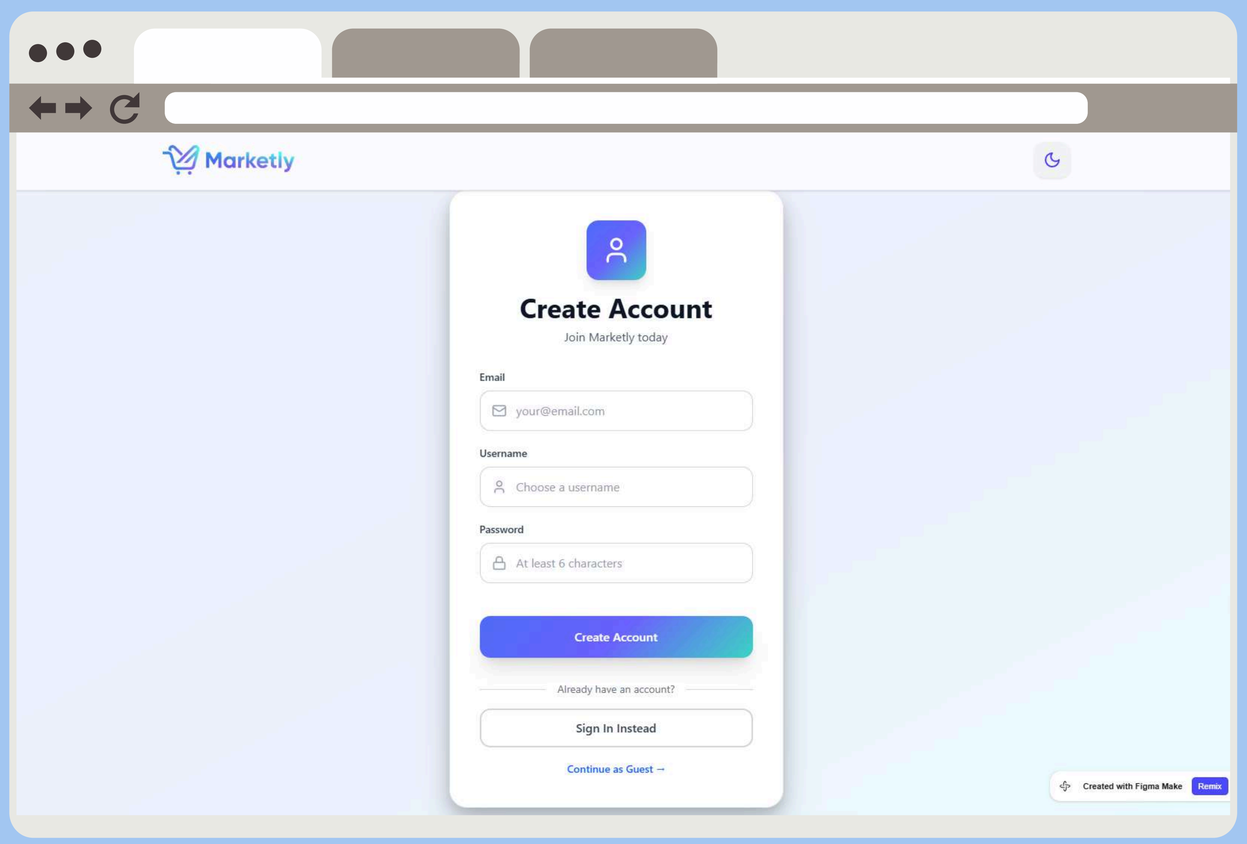
- Cart icon → shopping cart
- User icon → account

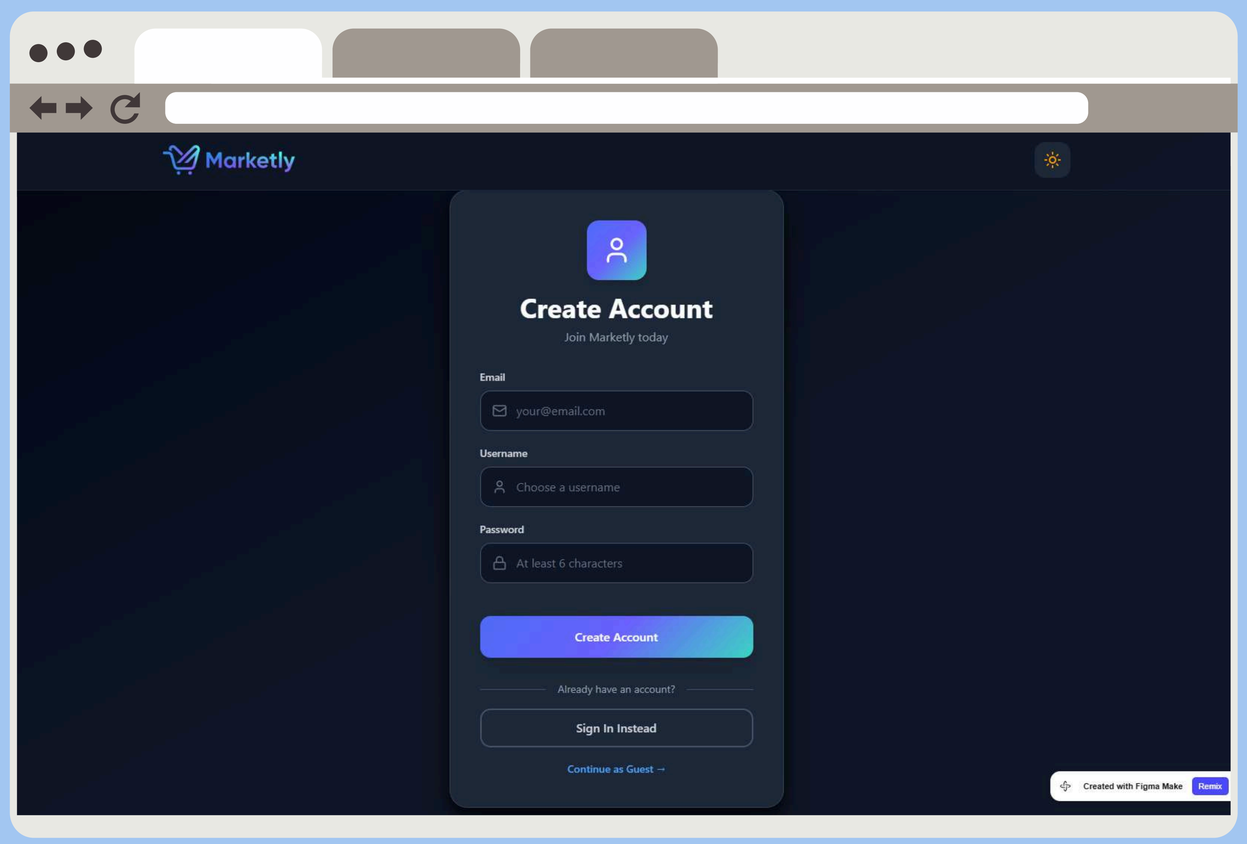
Step 4: Interface Design Prototyping

Used Technique: Figma / Web UI Design



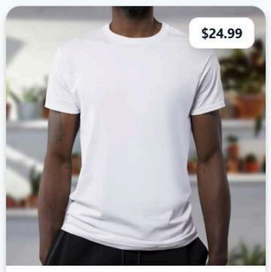




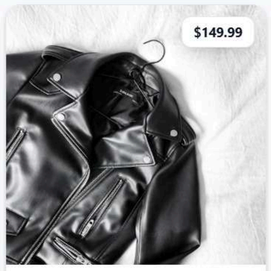


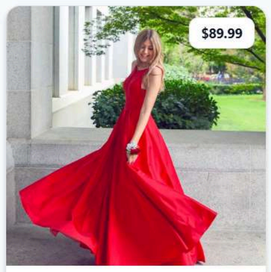
Marketly Home Clothes Electronics Furniture Shoes Miscellaneous Track Order admin 1

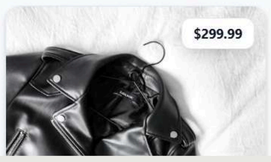
Featured Products


$24.99
Classic White T-Shirt

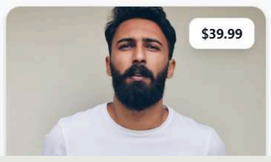

$79.99
Denim Jeans

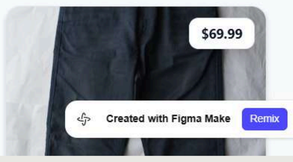

$149.99
Winter Jacket


$89.99
Summer Dress


$299.99


$54.99


$39.99


$69.99

Created with Figma Make Remix

Marketly Home Clothes Electronics Furniture Shoes Miscellaneous Track Order admin 1

Back

Hoodie

$54.99

Description

Comfortable cotton hoodie perfect for casual wear. Features adjustable drawstrings and front pocket.

Quantity

− 1 +

Add to Cart

Product Details

Category	Clothes
Product ID	#6
Availability	In Stock

Created with Figma Make Remix

Marketly Home Clothes Electronics Furniture Shoes Miscellaneous Track Order admin 1

Shoes

10 products found

Search products...

Filters

Price: Low to High

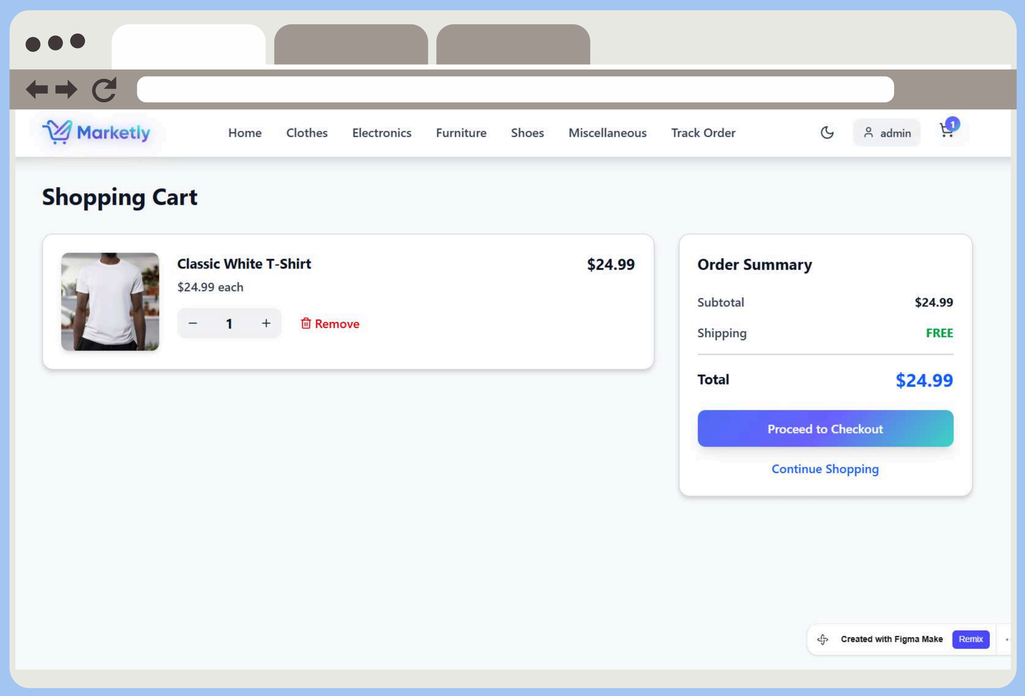

$49.99
Sandals

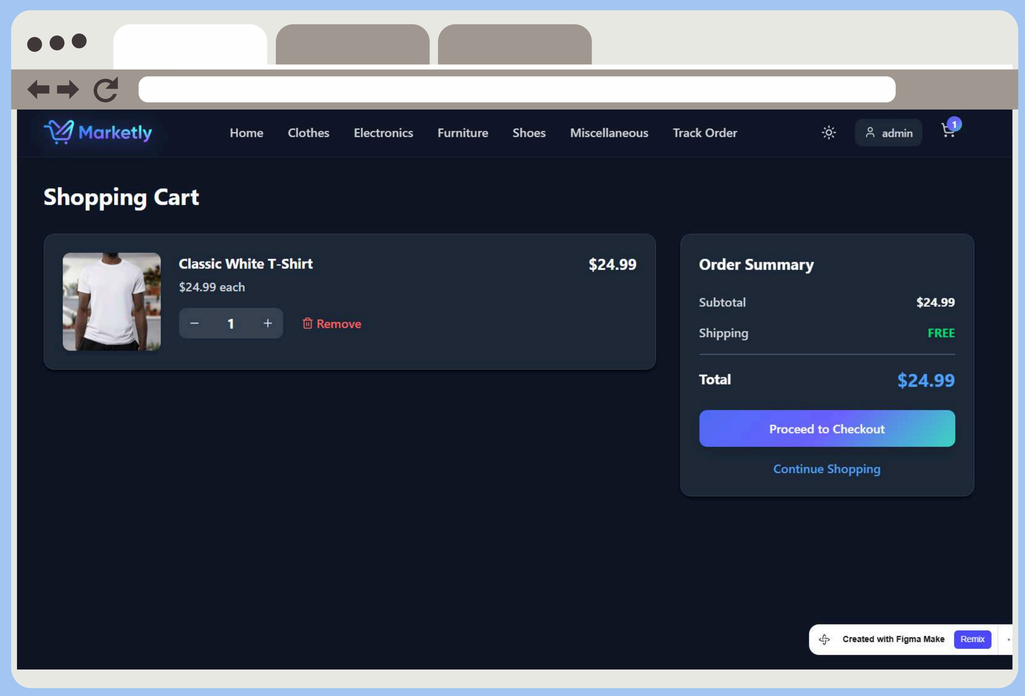

$64.99
Slip-on Sneakers

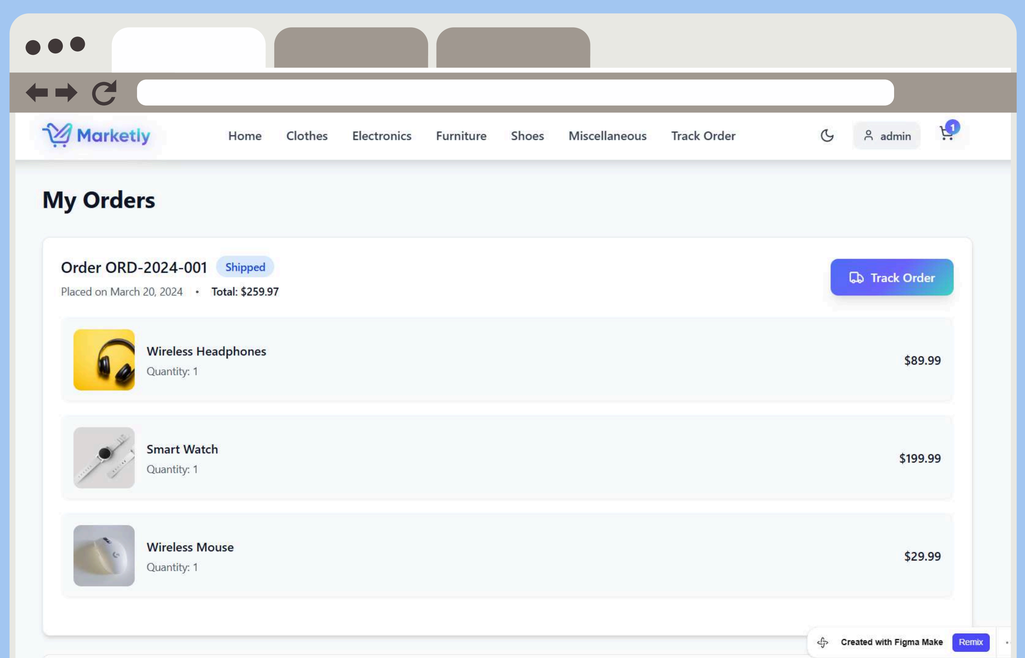

$79.99
Casual Sneakers

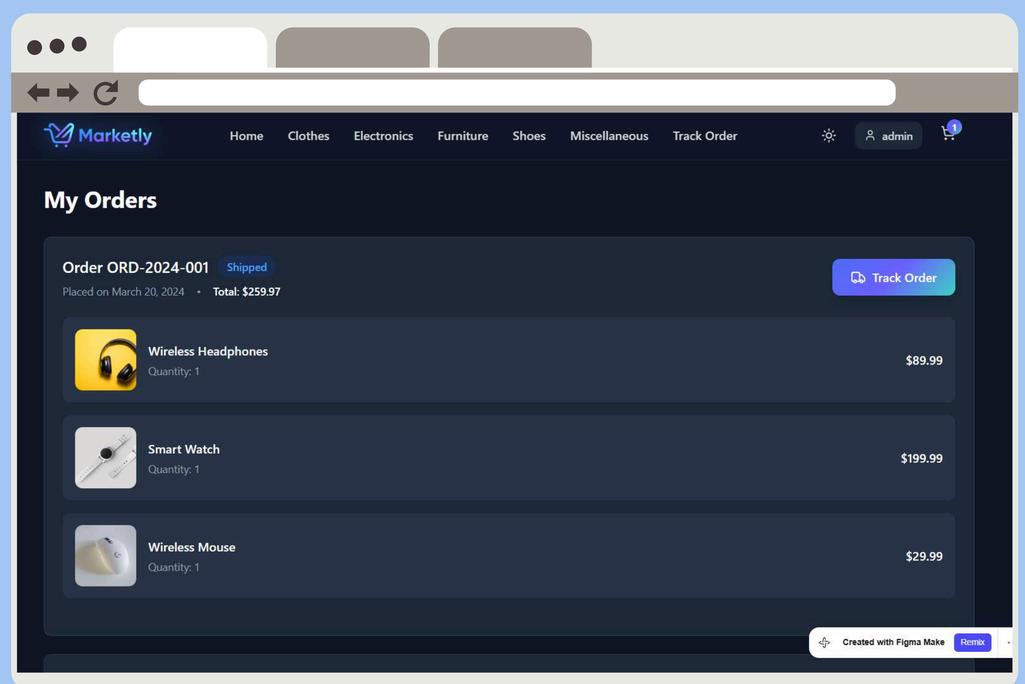

$89.99
Loafers

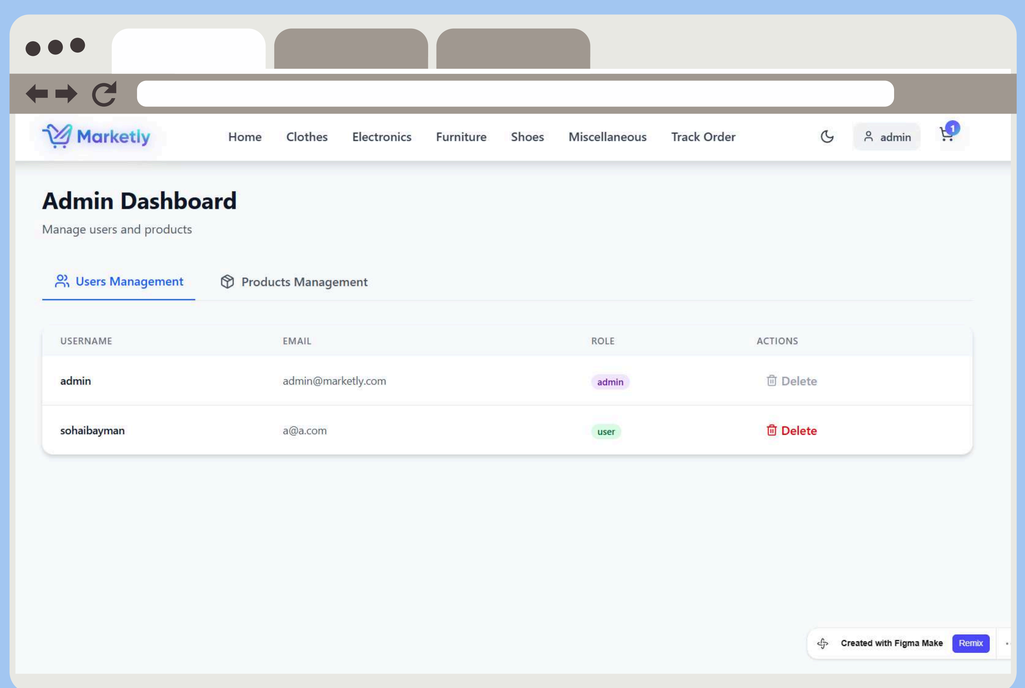
Created with Figma Make Remix

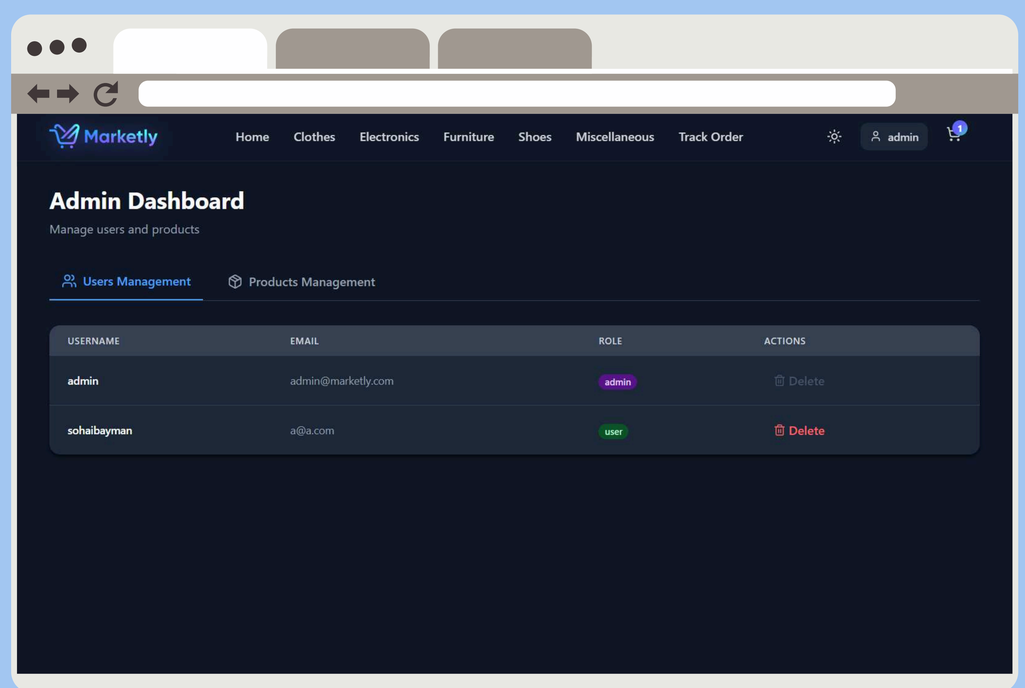

Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Shopping Cart
Classic White T-Shirt
$24.99 each
$24.99
1
Remove
Order Summary
Subtotal
$24.99
Shipping
FREE
Total
$24.99
Proceed to Checkout
Continue Shopping
Created with Figma Make
Remix

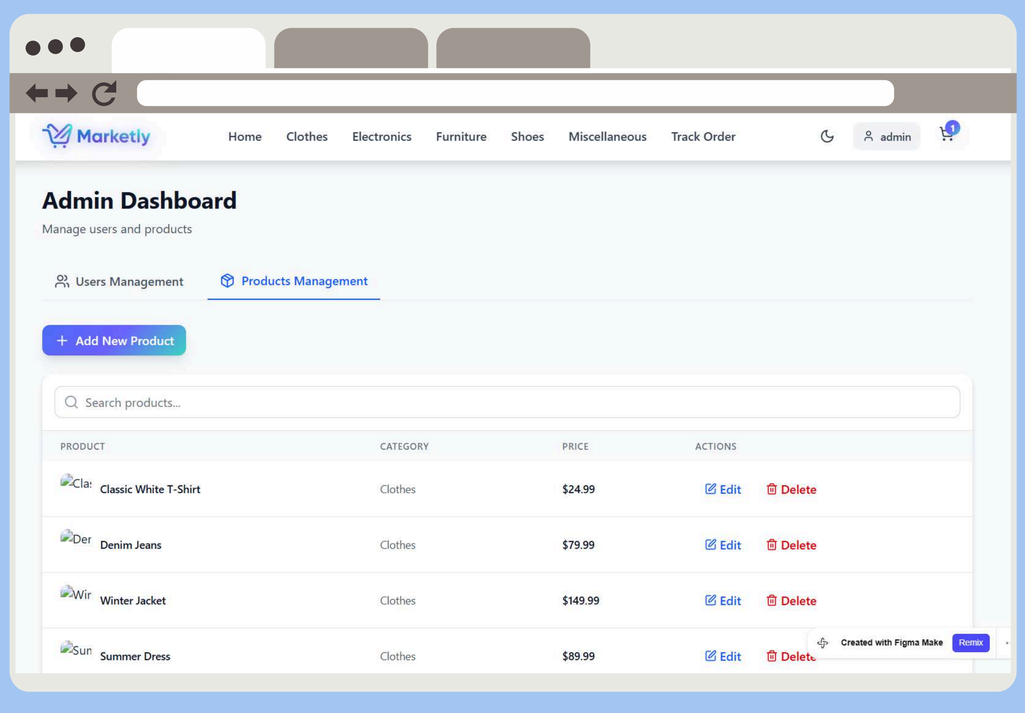

Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Shopping Cart
Classic White T-Shirt
$24.99 each
$24.99
1
Remove
Order Summary
Subtotal
$24.99
Shipping
FREE
Total
$24.99
Proceed to Checkout
Continue Shopping
Created with Figma Make
Remix

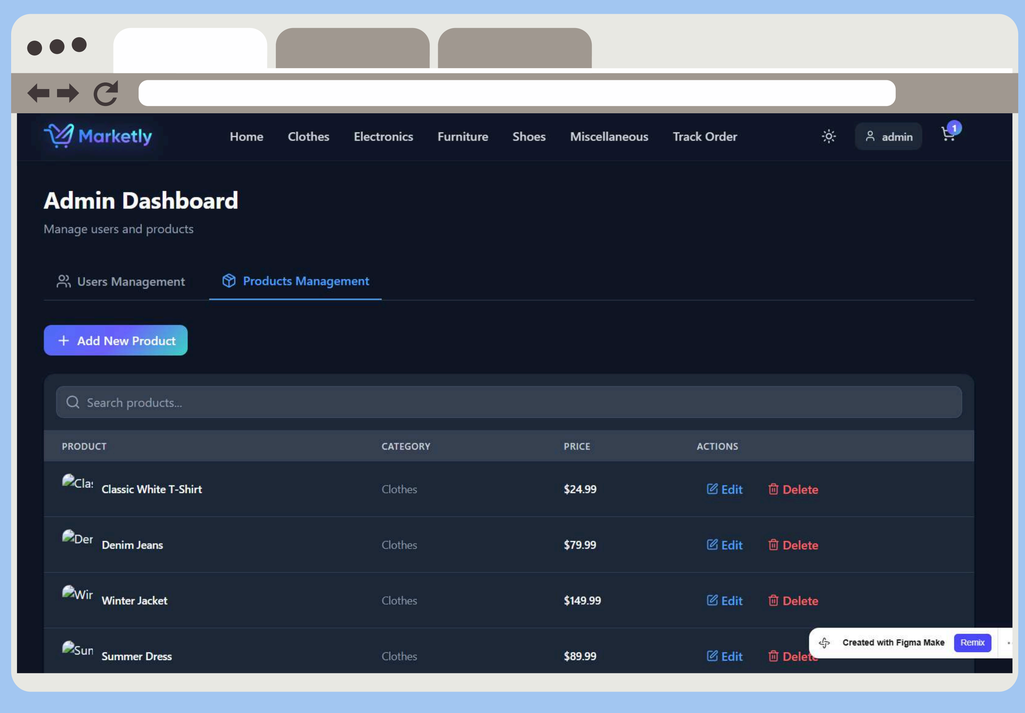

Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
My Orders
Order ORD-2024-001
Shipped
Placed on March 20, 2024
Total: $259.97
Track Order
Wireless Headphones
Quantity: 1
$89.99
Smart Watch
Quantity: 1
$199.99
Wireless Mouse
Quantity: 1
$29.99
Created with Figma Make
Remix


Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
My Orders
Order ORD-2024-001
Shipped
Placed on March 20, 2024
Total: $259.97
Track Order
Wireless Headphones
Quantity: 1
$89.99
Smart Watch
Quantity: 1
$199.99
Wireless Mouse
Quantity: 1
$29.99
Created with Figma Make
Remix


Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Admin Dashboard
Manage users and products
Users Management
Products Management
USERNAME
EMAIL
ROLE
ACTIONS
admin
admin@marketly.com
admin
Delete
sohaibayman
a@a.com
user
Delete
Created with Figma Make
Remix


Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Admin Dashboard
Manage users and products
Users Management
Products Management
USERNAME
EMAIL
ROLE
ACTIONS
admin
admin@marketly.com
admin
Delete
sohaibayman
a@a.com
user
Delete


Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Admin Dashboard
Manage users and products
Users Management
Products Management
Add New Product
Search products...
PRODUCT
CATEGORY
PRICE
ACTIONS
Classic White T-Shirt
Clothes
$24.99
Edit
Delete
Denim Jeans
Clothes
$79.99
Edit
Delete
Winter Jacket
Clothes
$149.99
Edit
Delete
Summer Dress
Clothes
$89.99
Edit
Delete
Created with Figma Make
Remix


Marketly
Home
Clothes
Electronics
Furniture
Shoes
Miscellaneous
Track Order
admin
Admin Dashboard
Manage users and products
Users Management
Products Management
Add New Product
Search products...
PRODUCT
CATEGORY
PRICE
ACTIONS
Classic White T-Shirt
Clothes
$24.99
Edit
Delete
Denim Jeans
Clothes
$79.99
Edit
Delete
Winter Jacket
Clothes
$149.99
Edit
Delete
Summer Dress
Clothes
$89.99
Edit
Delete
Created with Figma Make
Remix

Step 5: Interface Evaluation

Criteria	Status
Layout	✓
Content awareness	✓
Aesthetics	✓
User experience	✓
Consistency	✓
Minimize user effort	✓

Guidelines

1. Design Principles
    - The system incorporates all the modern features of good UI/UX design which helps users feel comfortable working with it:
        - Consistency: identical layout, space, components on each page
        - Visual Hierarchy: important elements like headings, prices, and buttons are easily distinguishable
        - User-centered approach: the interface is aimed at facilitating browsing and making purchase process easier
        - Feedback & Interactivity: hovering effects, buttons' states, updating cart
        - Role-Based Interface: administrative features are displayed only to authorized users.
        - Real-Time Data Integration: interface updates reflect Firestore data changes dynamically.

2. Color Scheme
    - Both Dark Mode and Light Mode are available.
        - Dark Mode:
            - Primary Gradient: 135deg, #4f6ef7, #6c63ff, #3ed6c5
            - Background: #f9fafb
            - Cards / Components: #f9fafb
            - Text: #0f172a
            - Accent: Gradient colors for buttons and highlights
        - Light Mode:
            - Primary Gradient: 135deg, #4f6ef7, #6c63ff, #3ed6c5
            - Background: #101828
            - Cards / Components: #1e2939
            - Text: #ffffff
            - Accent: Gradient colors for buttons and highlights

3. Typography
    - Font type: clean sans-serif font
    - Headings: big and bold
    - Body Text: average-size
    - Buttons / labels: brief and clear
    - Typography Hierarchy:
        - Big – titles
        - Medium – sections
        - Small – details

4. Accessibilities
    - Contrasting color palette in Dark and Light Modes
    - Dark and Light themes are supported
    - The system provides a responsive design that works well on all devices – mobile and computer
    - The system offers clear navigation
    - Clear and visible UI elements
    - Good spacing

5. Design of UI components
    - Buttons: rounded, gradient background, hover effect
    - Cards: minimalistic, shadowed
    - Navbar: minimalistic and consistent on all pages
    - Cart panel: distinct and structured

3.2 Technology Stack

- Frontend
  - React.js
  - HTML5
  - CSS3
  - JavaScript (ES6+)
  - Redux Toolkit
  - React Router DOM

- Authentication Services
  - Firebase Authentication
    - Used for user registration and login
    - Provides secure user authentication and session management

- Data Storage
  - Cloud Firestore
    - Used to store products
    - Used to store categories
    - Used to store users
    - Used to store orders
    - Used to store backup data

  - Browser LocalStorage
    - Used to store cart data
    - Used to store theme preferences

- External API
  - Platzi API
    - Used to import product data through the product seeding feature before storing it in Cloud Firestore

- Tool & Development Environment
  - Visual Studio Code
  - Git & GitHub
  - Figma (for UI/UX design)

- Database Services
  - Cloud Firestore
    - Provides cloud-based data storage
    - Supports real-time synchronization
    - Stores products, categories, users, orders, and backups

3.3 Additional Deliverables

1. API Documentation

- The system does not include a custom backend API. Instead, it is designed to connect with other systems, as follows:
    - Firebase Authentication for user registration and login processes.
    - Cloud Firestore will store products, categories, users, orders, and backup data.
    - Platzi API for product seeding operations.
- The application will communicate with Firebase services using the Firebase SDK.

2. Testing & Validation

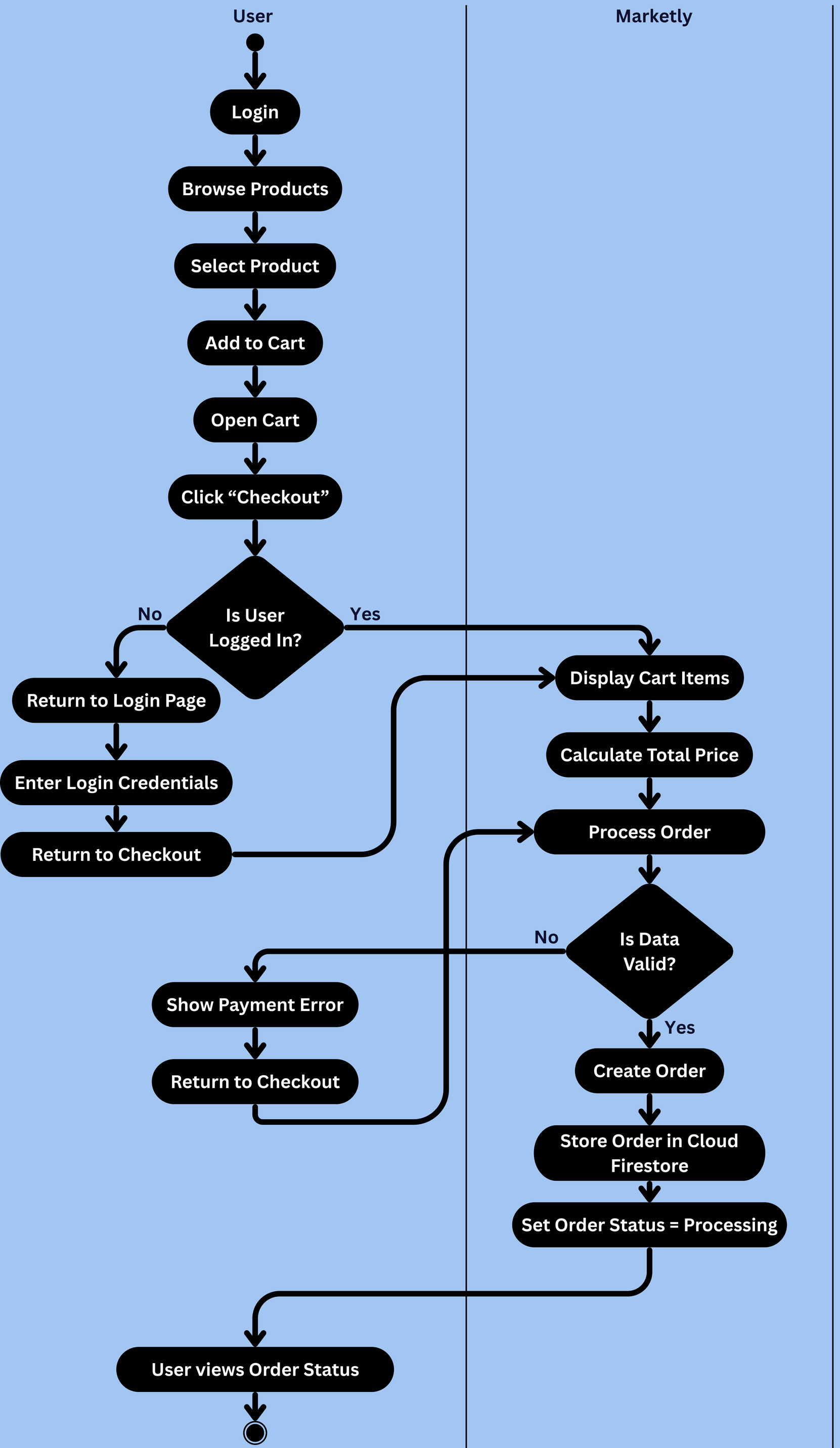
- The system will be tested in several ways in order to validate its reliability and correctness:
    - Unit Testing: Core functionalities such as product listing, cart operations, authentication, and order processing are tested individually.
    - Integration Testing: Interactions between React components, Firebase Authentication, Cloud Firestore, Redux Toolkit, and routing modules are validated.
    - User Acceptance Testing (UAT): The system is tested from both customer and administrator perspectives to ensure all business requirements are satisfied.

3. Deployment Strategy

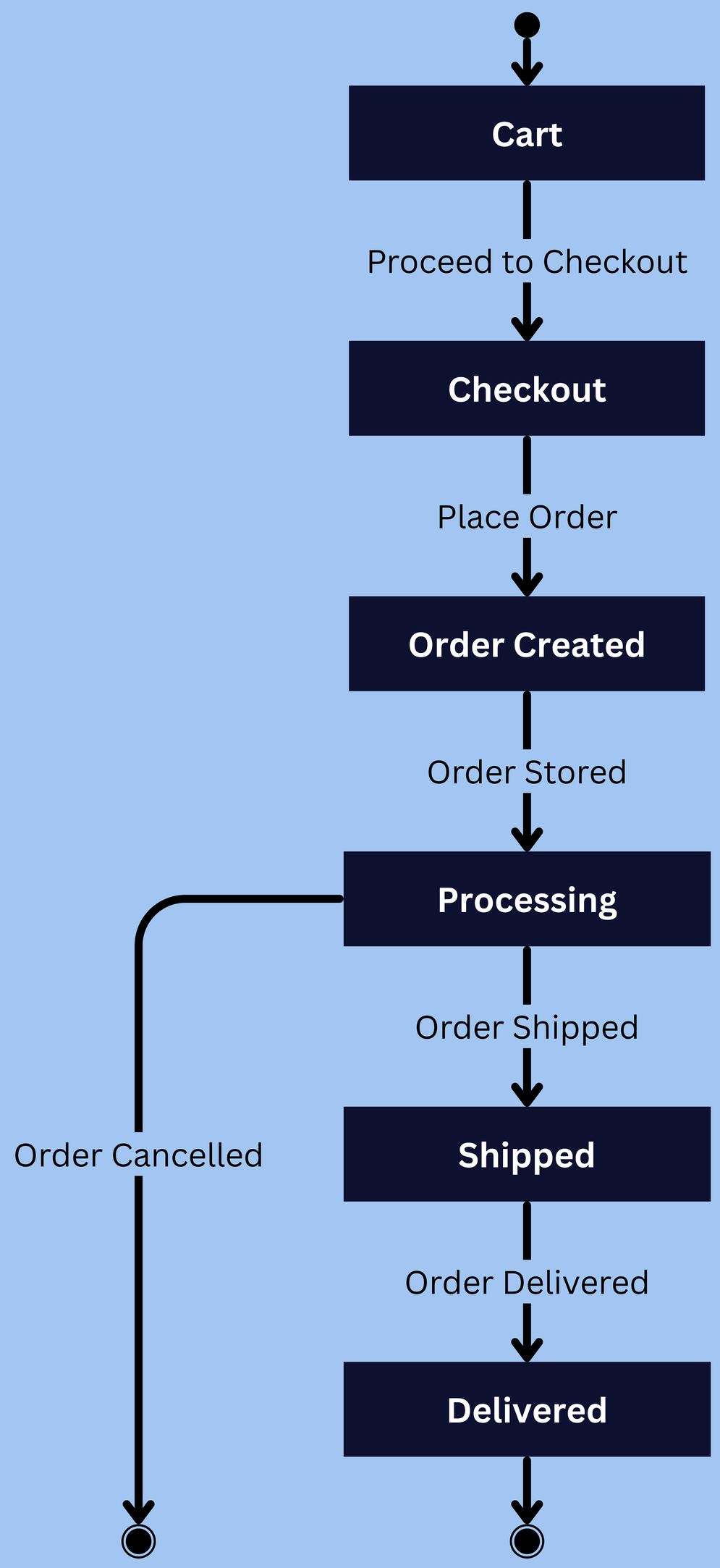
- As a web application, this project is designed to be deployed as follows:
    - Hosting: The application will be deployed using Vercel.
    - Build process: The react-based application will be created using the appropriate build tooling (e.g., `npm build`).
    - Scalability: Firebase Authentication and Cloud Firestore provide scalable cloud infrastructure capable of supporting future growth.
    - Availability: The system will be accessed from the Web via browsers.

3.4 UML Diagrams

3.4.1 Activity Diagram - Place Order



3.4.2 State Diagram - Order Lifecycle

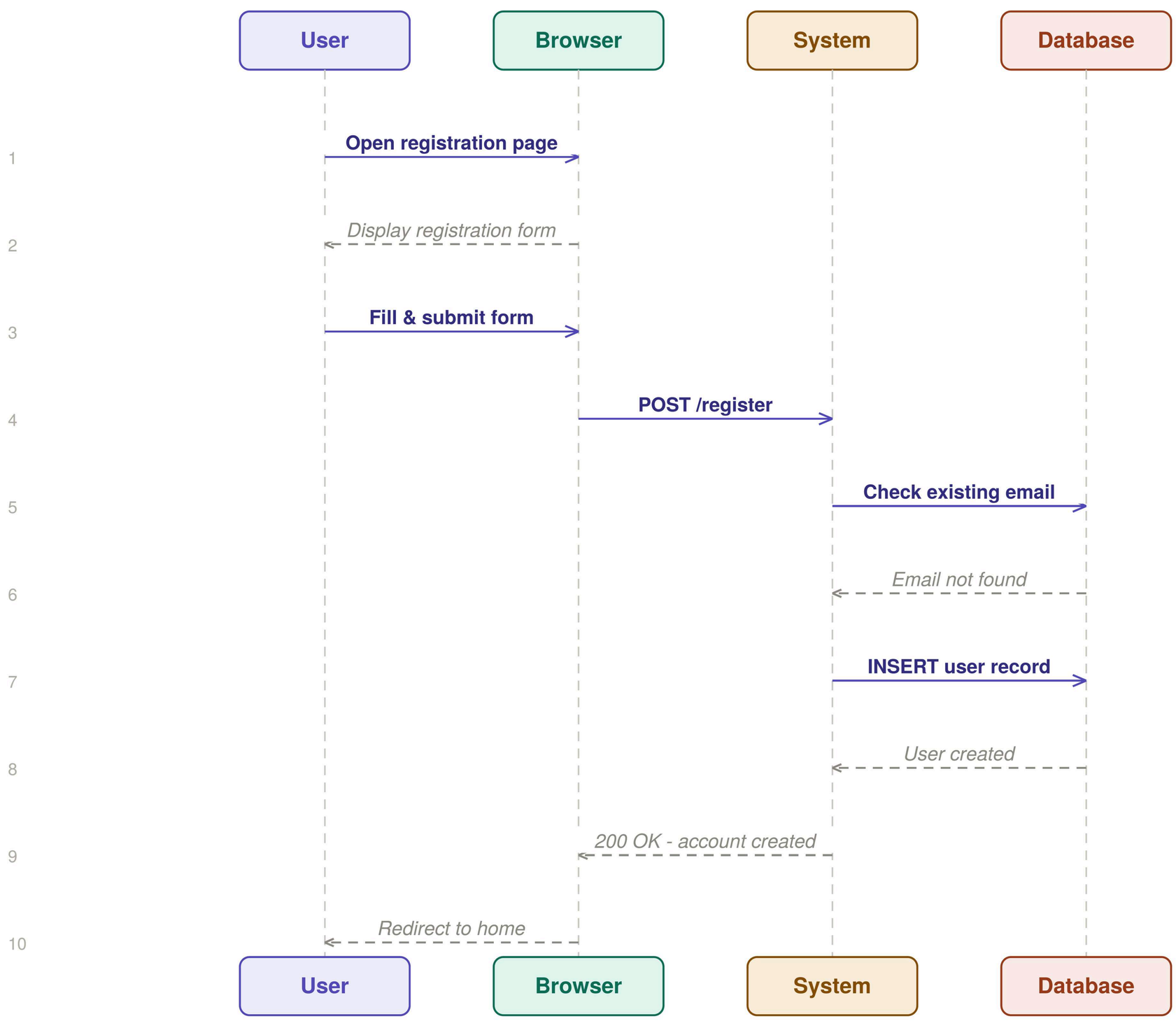


3.4.3 Sequence Diagram

UC-1 : Register

UC-1 **Register** Customer Medium Priority

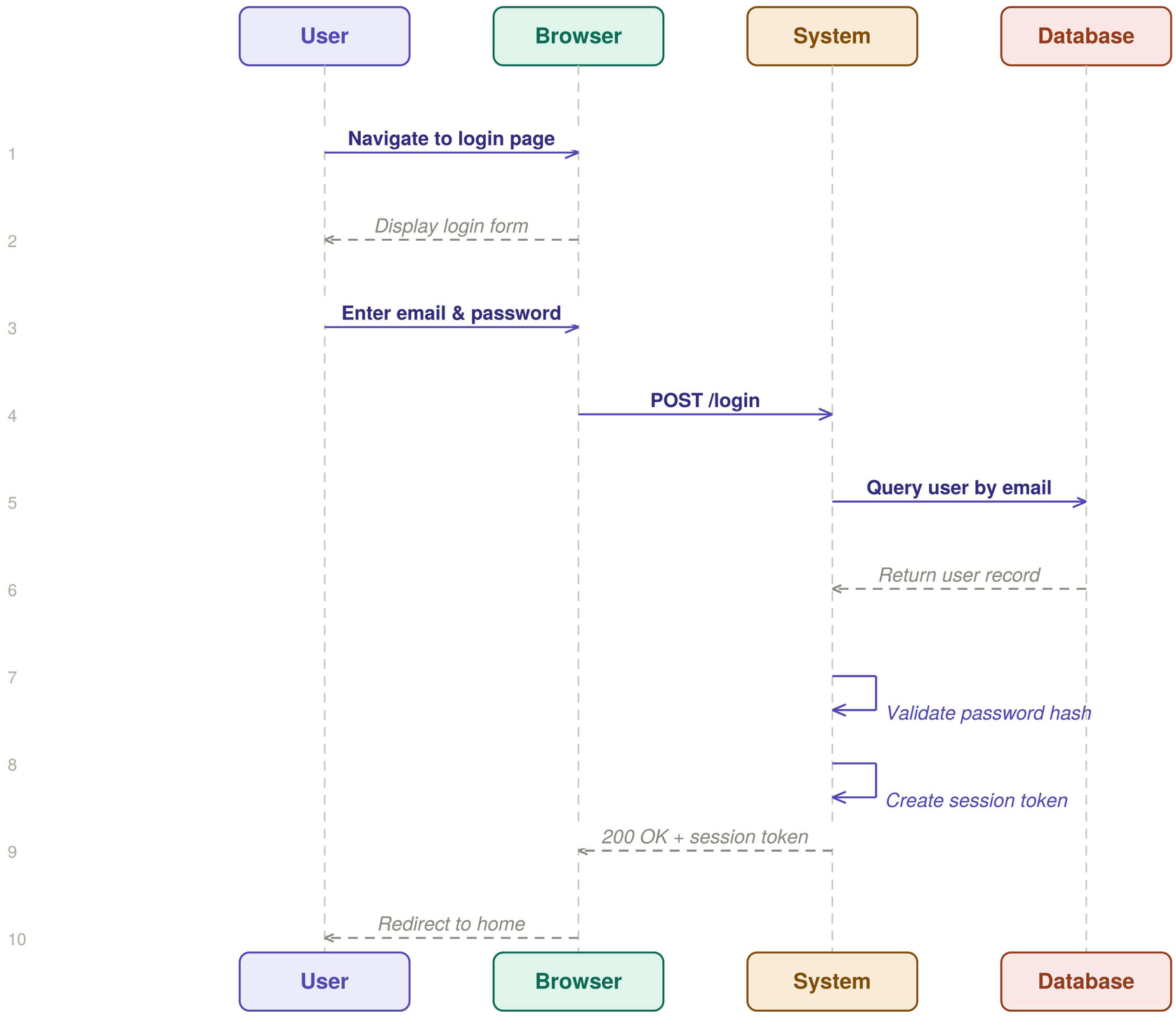
New user creates an account on Marketly.



UC-2: Login

UC-2 **Login** Customer High Priority

Registered user logs into their account.



UC-3: Browse Categories

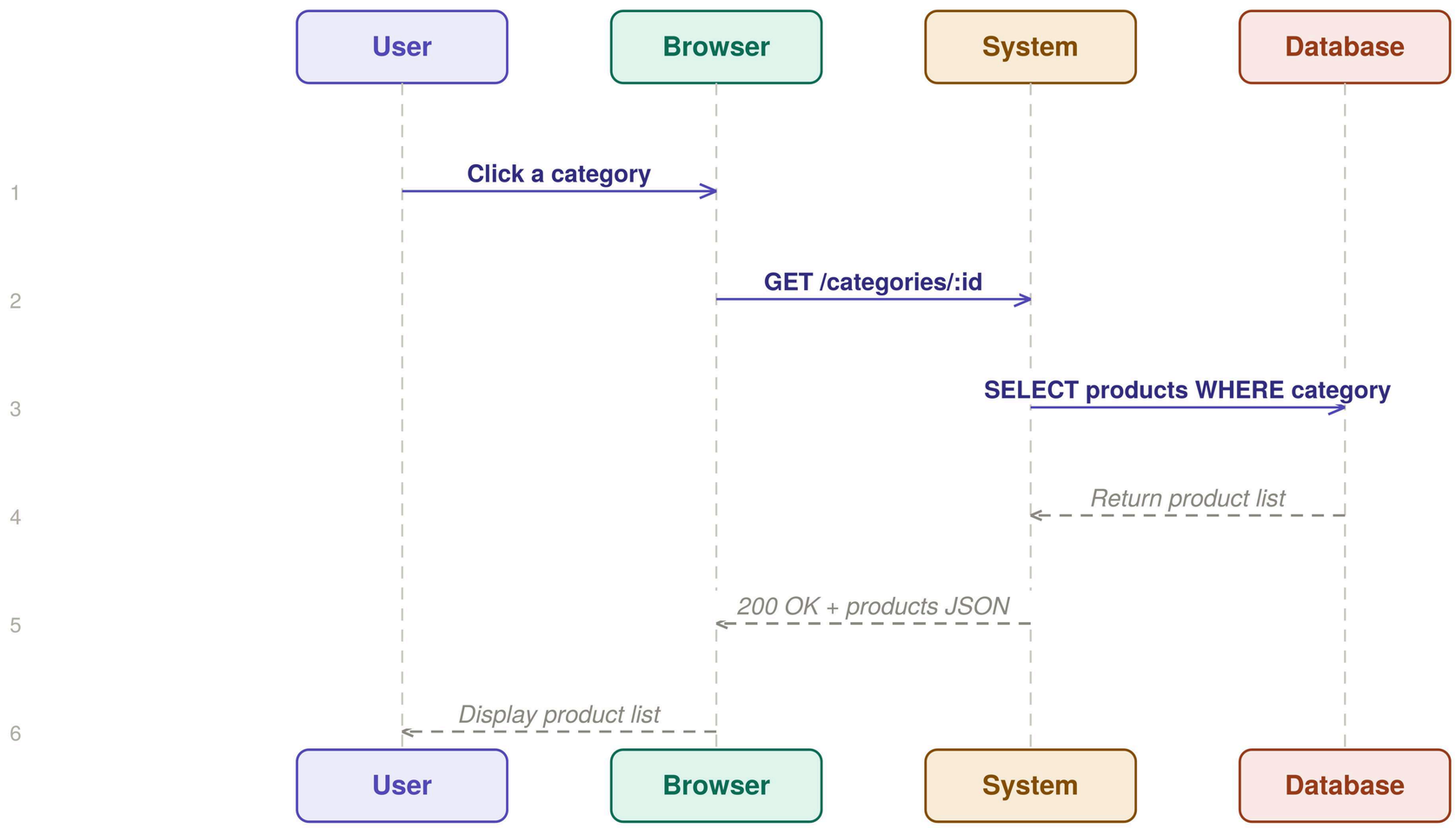
UC-3

Browse Categories

Customer

High Priority

User browses product categories to find items.



UC-4: Search Products

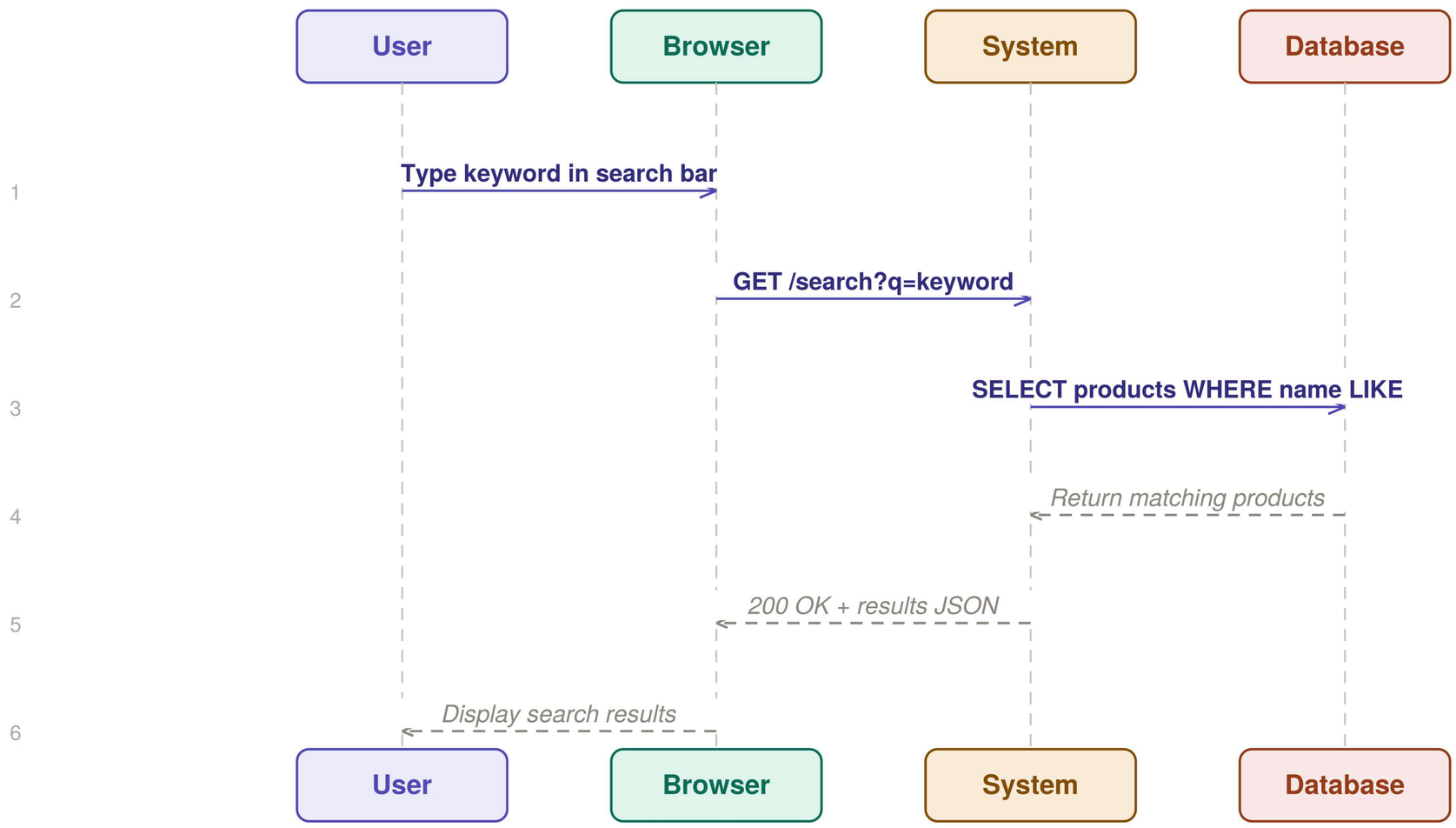
UC-4

Search Products

Customer

High Priority

User searches for products using a keyword.



UC-5: View Product Details

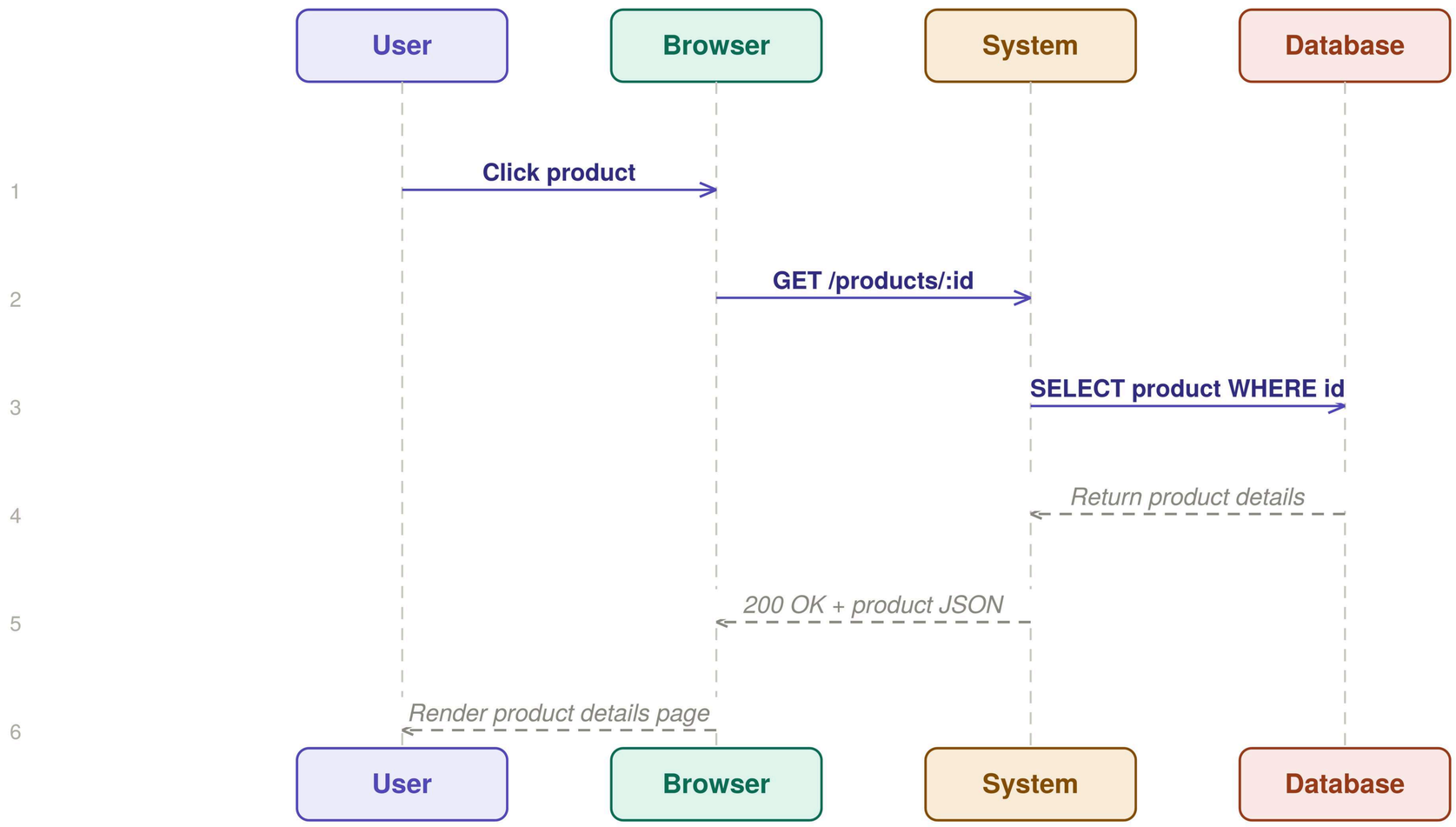
UC-5

View Product Details

Customer

High Priority

User views full details of a selected product.



UC-6: Filter by Price

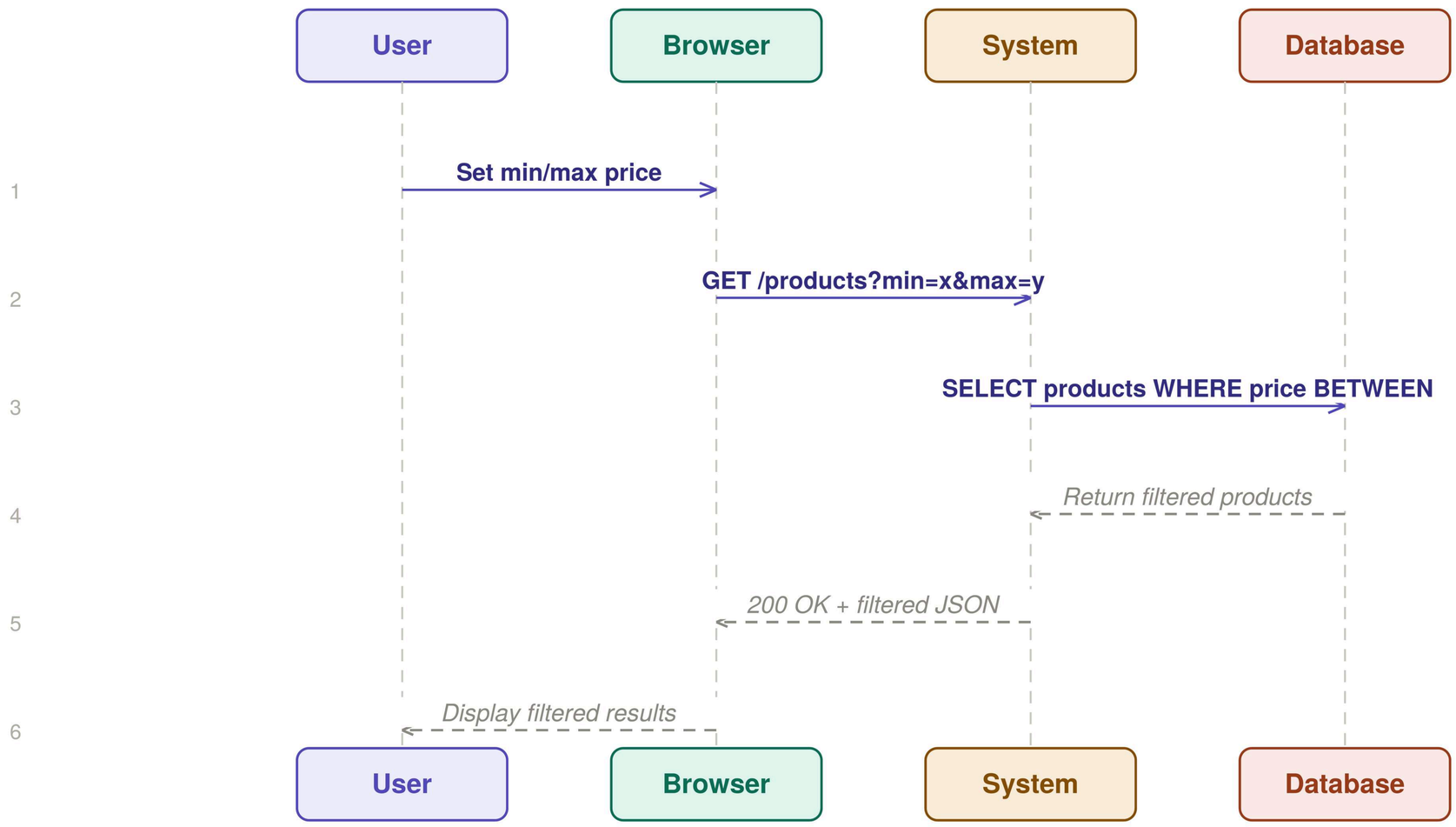
UC-6

Filter by Price

Customer

Medium Priority

User filters products by a price range.



UC-7: Sort by Price

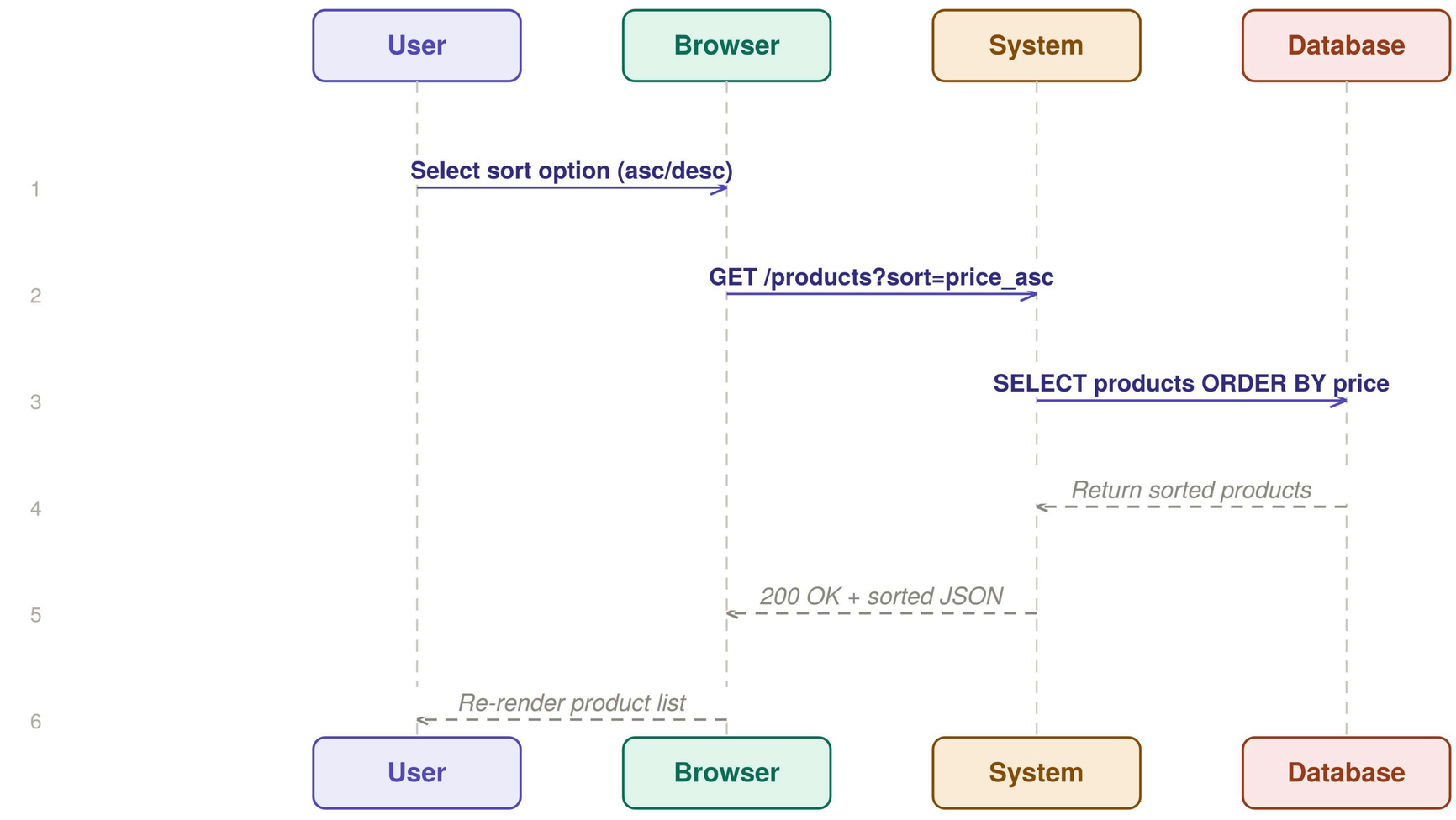
UC-7

Sort by Price

Customer

Low Priority

User sorts products by price order.



UC-8: Add to Cart

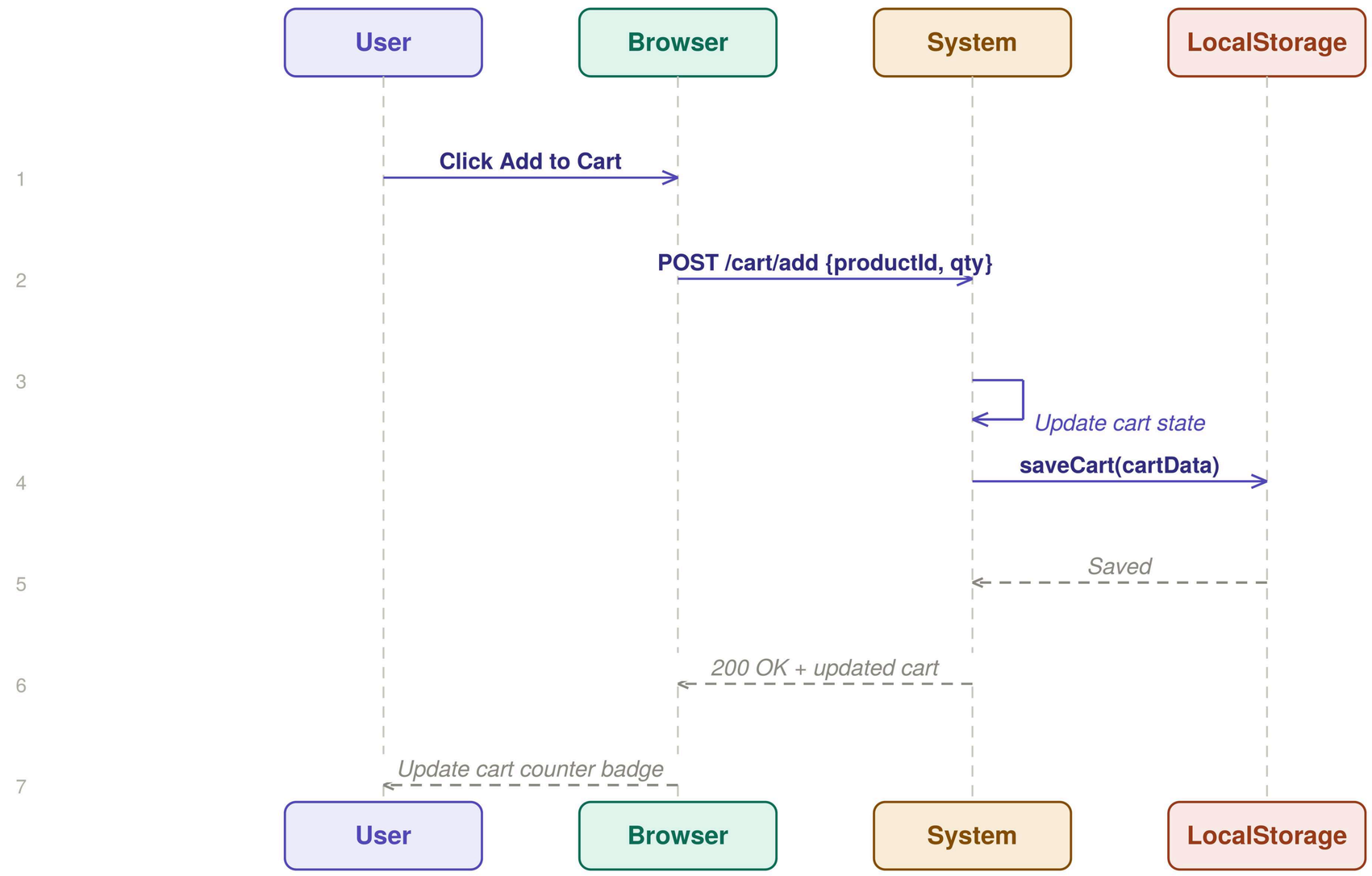
UC-8

Add to Cart

Customer

High Priority

User adds a product to the shopping cart.



UC-9: Remove from Cart

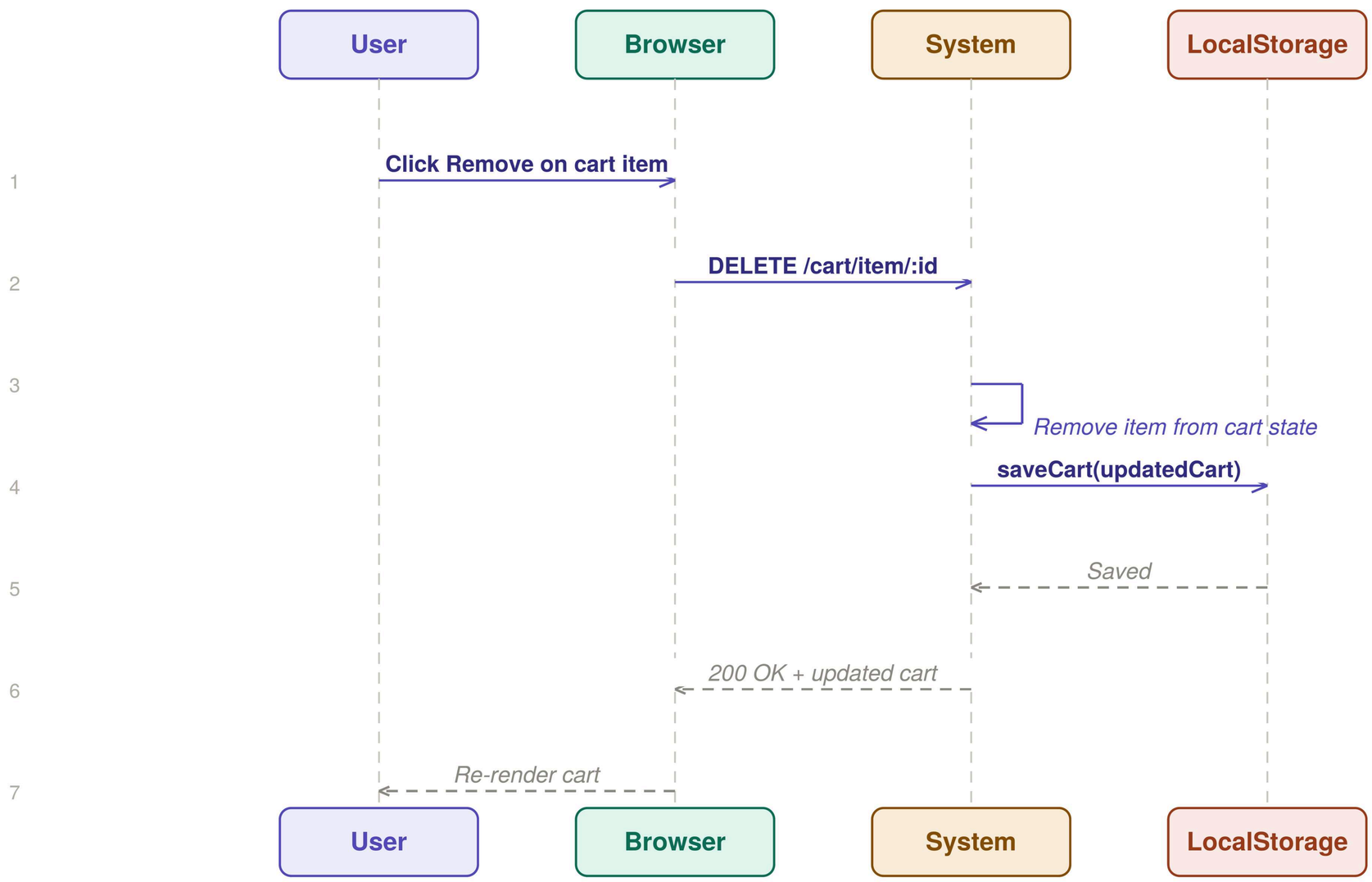
UC-9

Remove from Cart

Customer

Medium Priority

User removes a product from the cart.



UC-10: Change Quantity

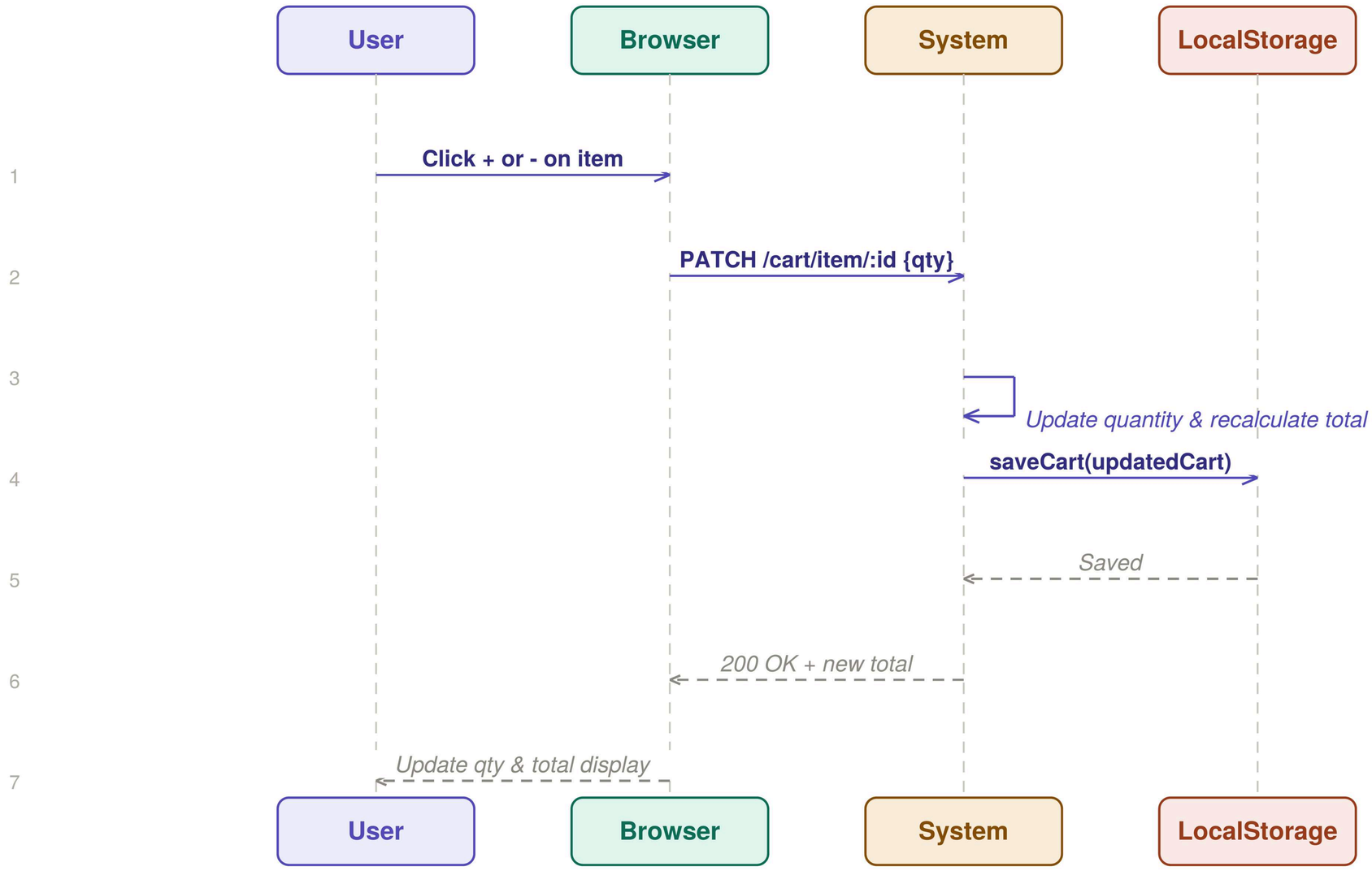
UC-10

Change Quantity

Customer

Medium Priority

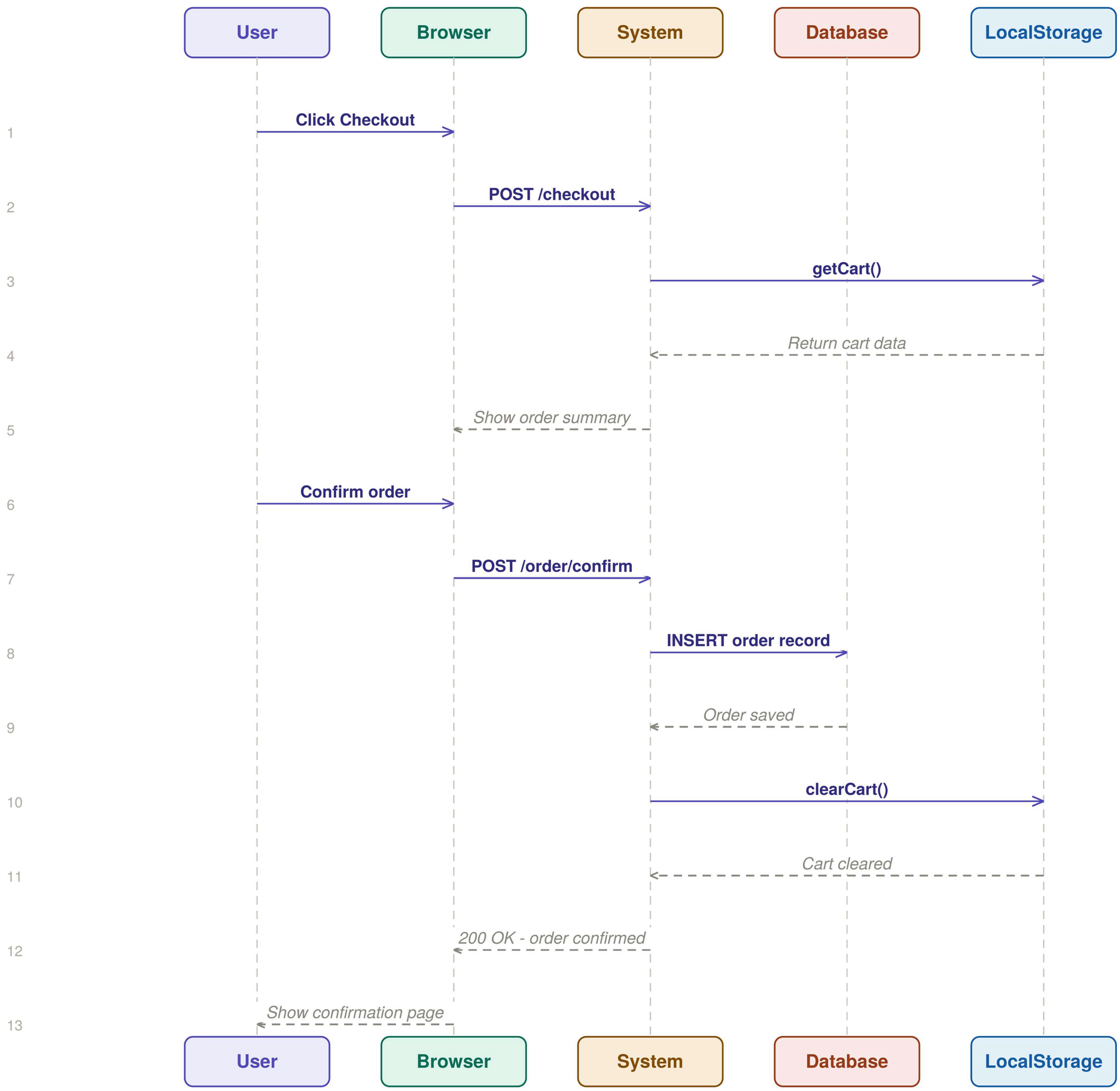
User changes the quantity of a cart item.



UC-11 :Checkout

UC-11 **Checkout** Customer High Priority

Logged-in user completes a purchase.



UC-12: Manage Products

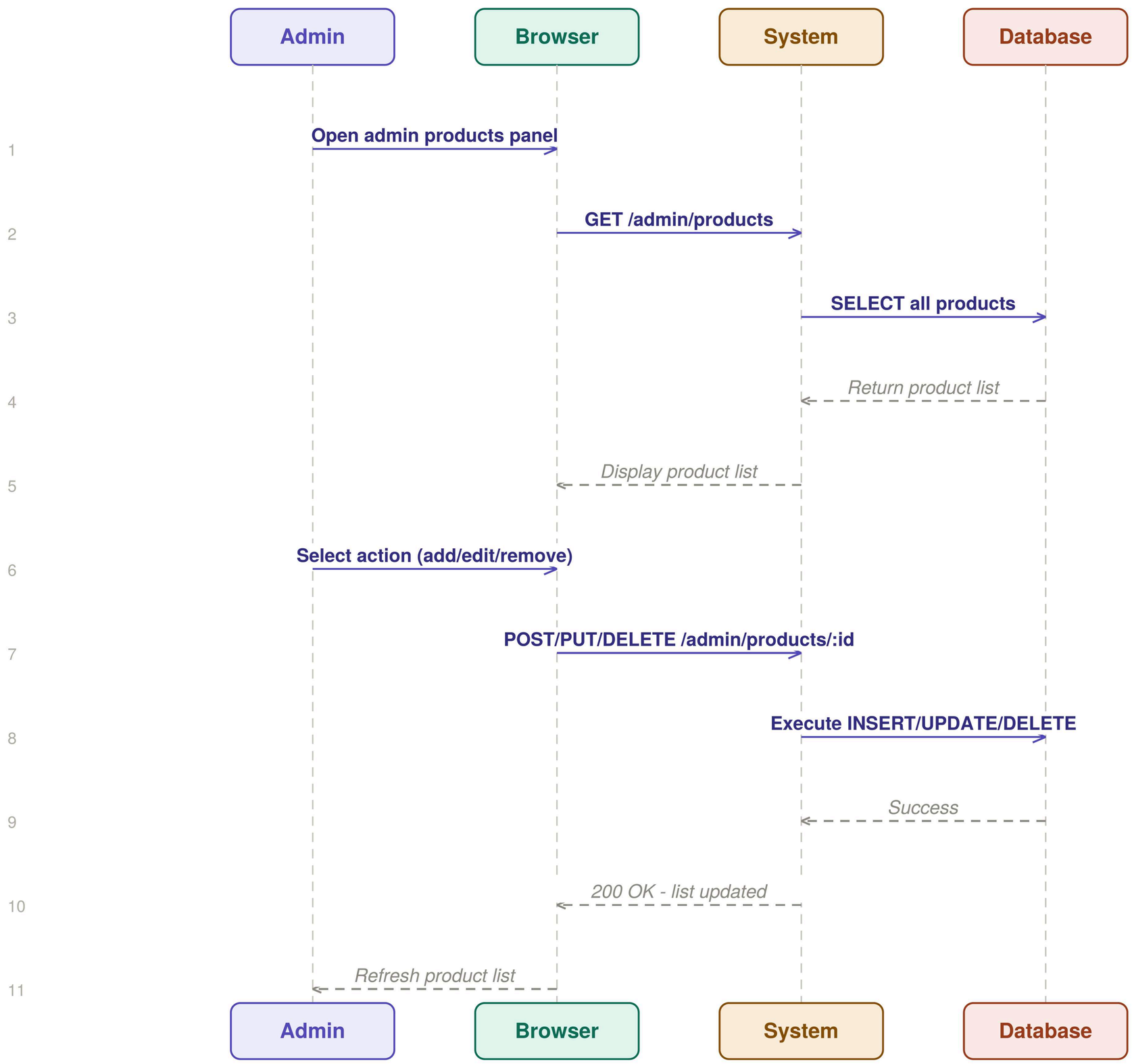
UC-12

Manage Products

Admin

High Priority

Admin adds, edits, or removes products from the system.



UC-13: Manage Users

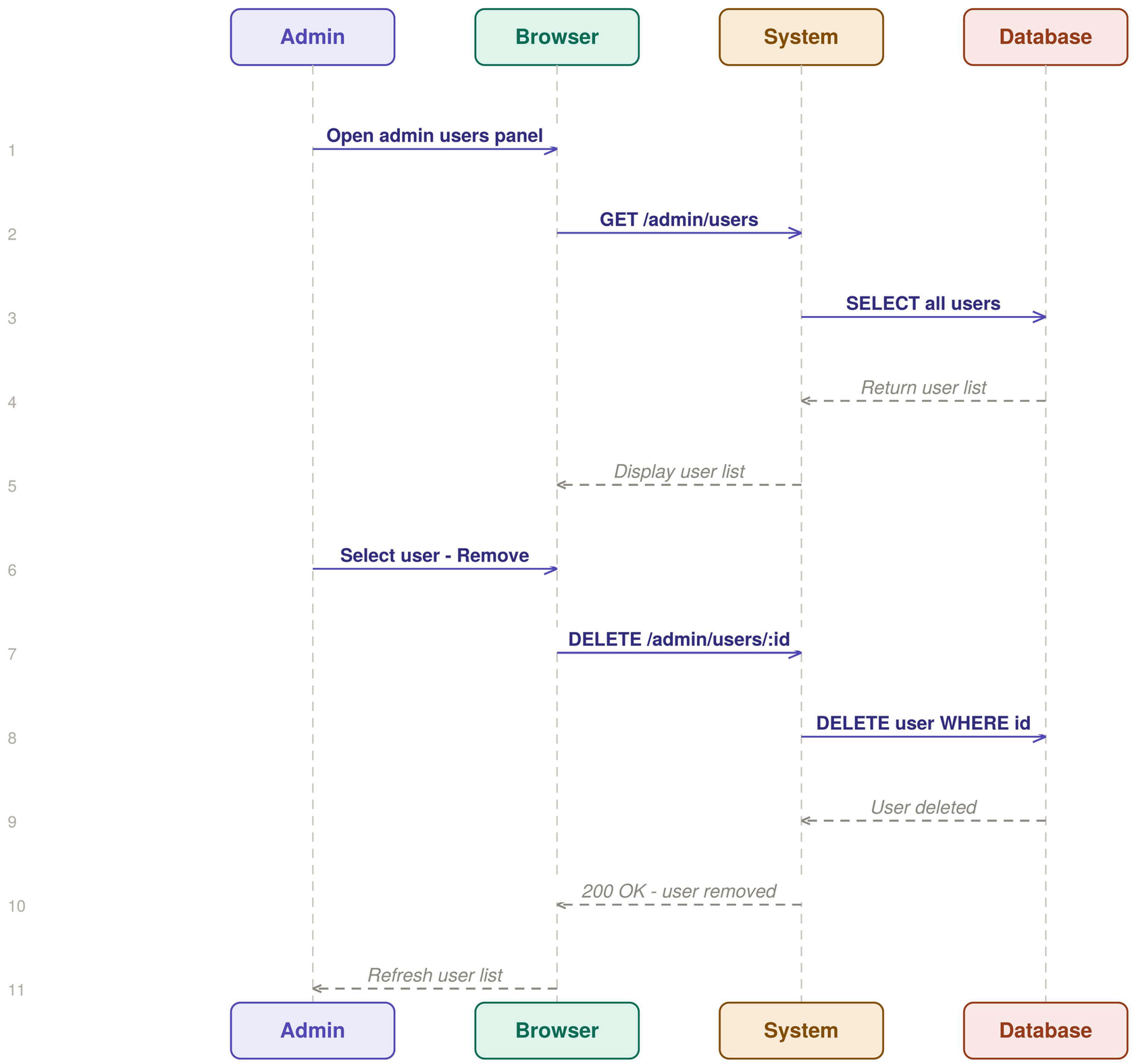
UC-13

Manage Users

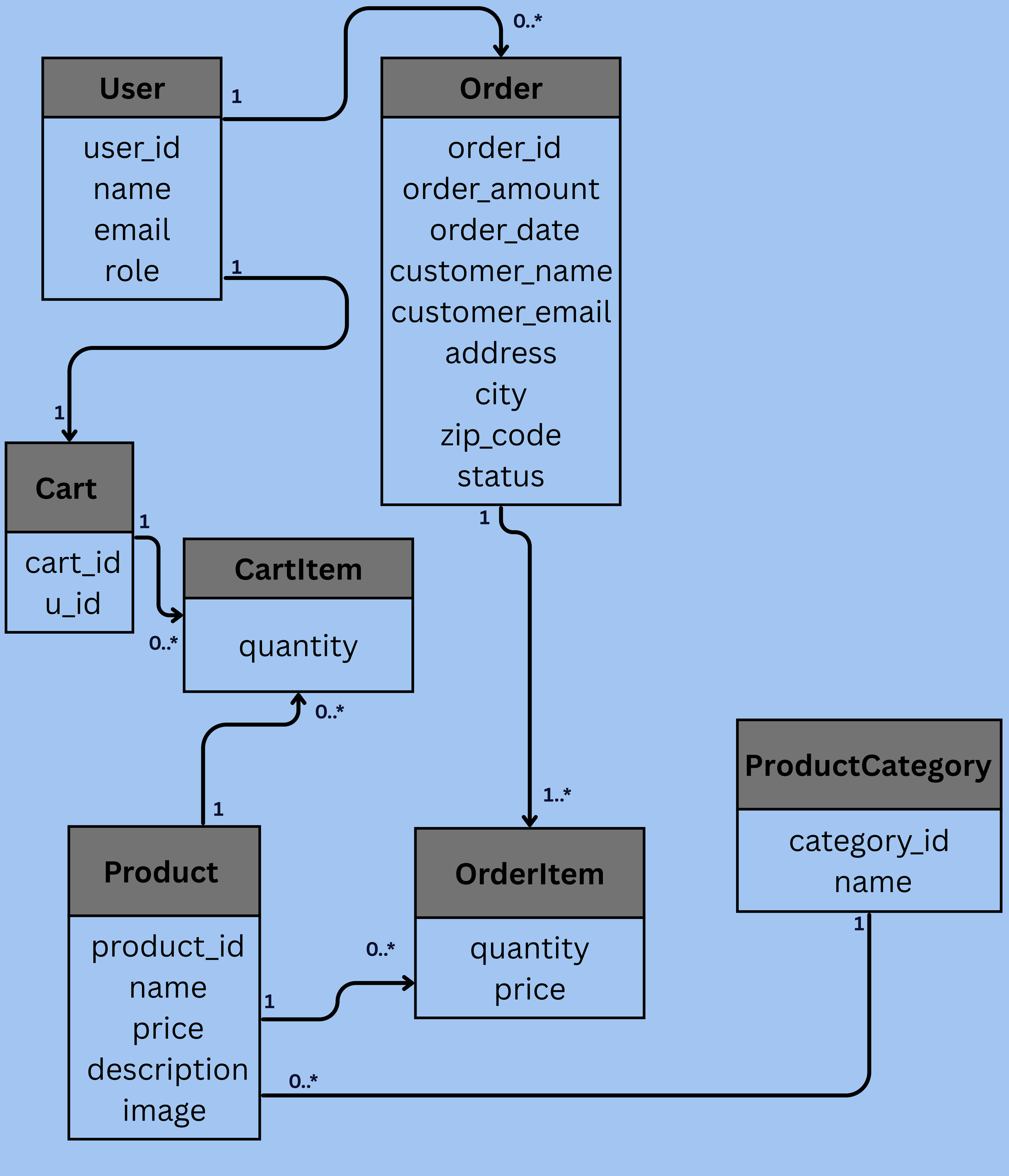
Admin

Medium Priority

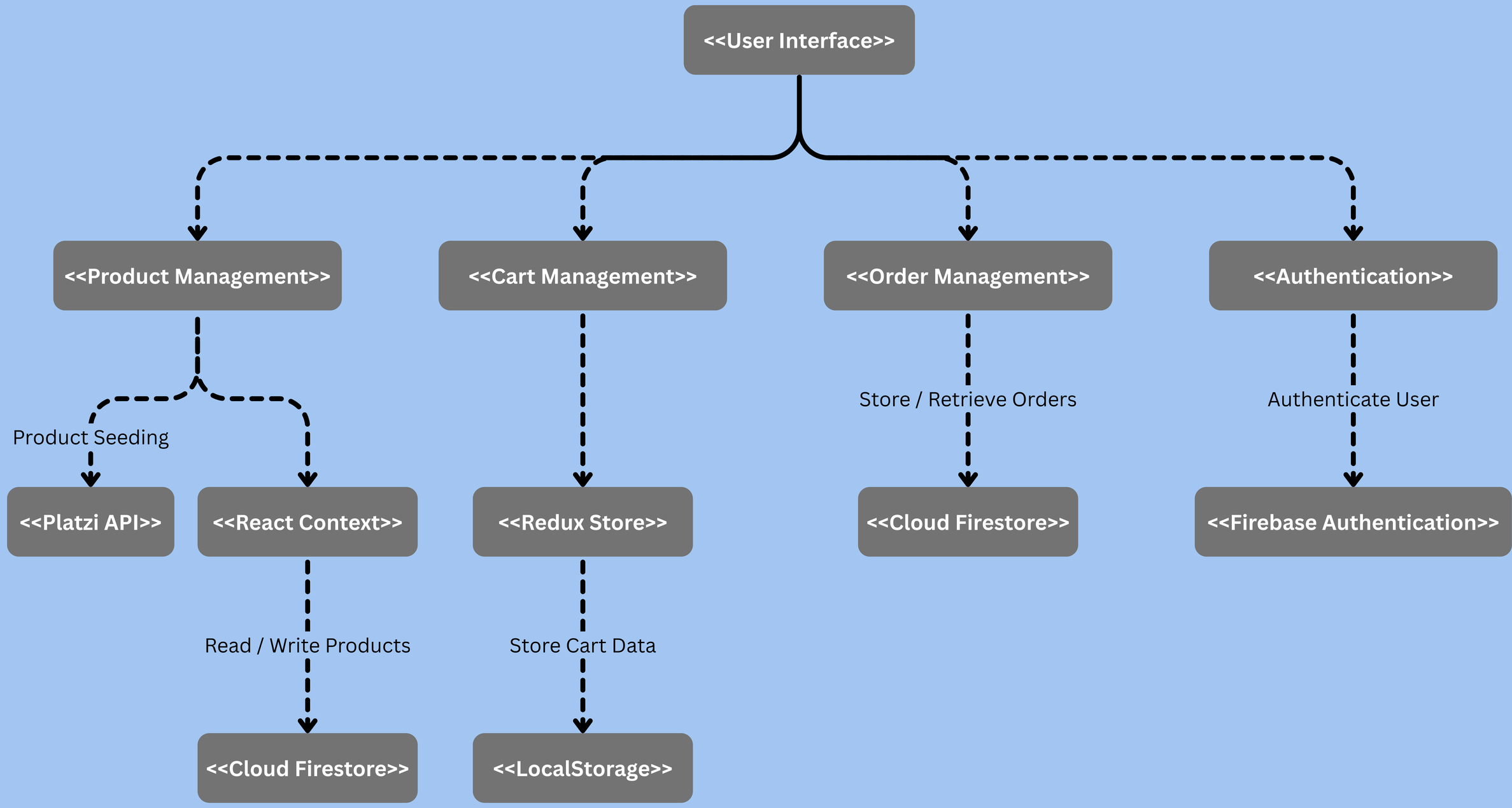
Admin views and removes registered users.



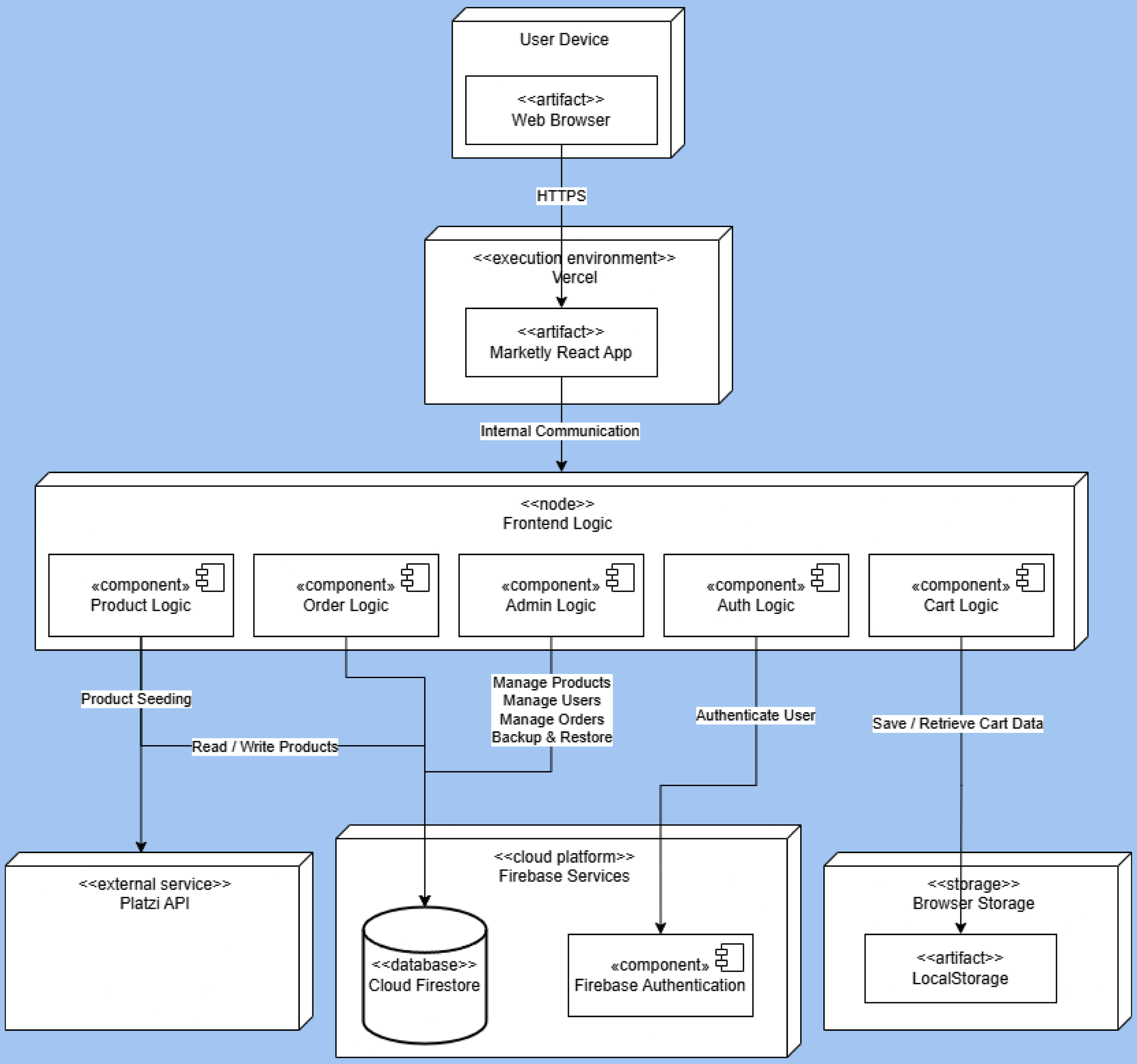
3.4.4 Class Diagram



3.4.5 Component Diagram



3.4.6 Deployment Diagram



IMPLEMENTATION

CODE EDITOR

HTML

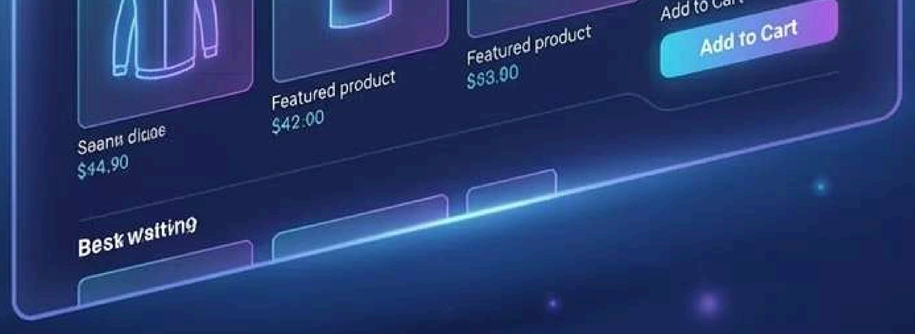
```
<html>
  <meta class="HTF-8">
  <meta class="testesr">
  <tite name="zonzet">Chuiet</title>
  <title clas="contnents"?></title>
</head>
</html>
```

CSS

```
<div class="commerce-sheet">
  <div class="utf-8">
  <div class=="eodaet"/>
  <div class="test6></div>
</div>
</div>
```

JavaScript

```
<script type="readsrer">
  <cript {
    <thsx clas="javaScript.class") {
    <thsx class="javaScript.cem");
  }
</script>
</script>
```



API CONNECTION DIAGRAM

DATABASE SCHEMAS

PRODUCTS

id
name
price
price
status
status

USERS

id
email
status

ORDERS

id
name
email
status

DEPLOYMENT PROGRESS

4.1 Source Code

4.1.1 Frontend Implementation

The frontend of Marketly was developed using React.js as the main JavaScript library for building the user interface. The application follows a component-based architecture, which improves maintainability, scalability, and code reusability.

React Router DOM was used to implement client-side routing, enabling smooth navigation between pages without requiring full page reloads. The user interface was designed to provide a modern and responsive shopping experience across different screen sizes and devices.

The frontend includes several core modules, such as product browsing, category pages, product details, shopping cart management, authentication pages, order tracking, and the administrative dashboard. Each module was implemented as a collection of reusable React components to simplify future maintenance and feature expansion.

State management was implemented using Redux Toolkit and React Context API. Redux Toolkit manages cart-related operations, while Context API handles user authentication and application data synchronization.

CSS Modules and Bootstrap were used to create a responsive and consistent design system. Dark Mode and Light Mode support were also implemented to enhance user experience and accessibility.

To improve search engine visibility, React Helmet Async was integrated for dynamic page titles and metadata management. Additional user feedback mechanisms were implemented using React Hot Toast to provide real-time notifications for important actions such as adding products to the cart, completing orders, and updating administrative data.

The frontend communicates with Firebase services in real time, allowing users and administrators to interact with up-to-date data without requiring manual page refreshes.

4.1.2 State Management and Data Handling

State management was implemented using a combination of Redux Toolkit and React Context API to ensure efficient data sharing and synchronization across the application. Redux Toolkit was used to manage shopping cart operations, including adding products, removing products, updating quantities, and persisting cart data between user sessions. This approach simplified complex state updates and improved overall application performance.

React Context API was used to manage global application data such as authenticated user information and product-related data. Dedicated context providers were created to centralize data access and reduce unnecessary prop drilling across components.

The UserContext is responsible for authentication state management, user role assignment, guest user handling, and synchronization with Firebase Authentication. It also supports anonymous authentication for users who browse the platform without creating an account.

The DataContext manages product and category data retrieved from Cloud Firestore. Real-time listeners were implemented using Firestore snapshot subscriptions, allowing the application to automatically reflect database changes without requiring manual refreshes.

This architecture separates business logic from presentation components, resulting in improved maintainability, scalability, and code organization. The combination of Redux Toolkit and Context API provides a balanced solution for handling both application-wide state and feature-specific data.

4.1.3 Backend Service Integration

Marketly utilizes Firebase as the primary backend platform, providing authentication services, cloud database functionality, and real-time data synchronization. Firebase Authentication was integrated to support user registration, login, logout, and session management. The system supports both authenticated users and anonymous users, allowing visitors to browse products and build shopping carts before creating an account.

Cloud Firestore serves as the main database for storing and managing application data. Multiple collections were implemented to organize products, categories, users, orders, and backup data. Firestore provides real-time synchronization, ensuring that changes made by administrators are immediately reflected across the application.

The order management system was fully integrated with Firestore. Customer orders are stored in the database, allowing users to access their order history and track order status across different devices. Administrative users can monitor and update order statuses through the dashboard interface.

Role-based access control was implemented through Firestore user records. Different permission levels were assigned to standard users, administrators, and owners, enabling controlled access to management features and sensitive operations.

Additional backend functionality includes product seeding from external APIs and a backup and restore mechanism for critical data collections. These features improve maintainability and support efficient system administration.

The integration between React and Firebase was designed to provide secure, scalable, and real-time communication between the frontend application and backend services.

4.1.4 Security and Error Handling

Security considerations were incorporated throughout the development of Marketly to protect application data and ensure controlled access to system resources.

Firebase Authentication was used to verify user identities and manage secure access to protected features. Authentication mechanisms ensure that only authorized users can access personal data, order information, and administrative functionality.

Role-based access control was implemented to differentiate between standard users, administrators, and owners. Each role is assigned a specific set of permissions, limiting access to sensitive operations such as product management, user management, backup operations, and system administration features.

Cloud Firestore security rules were configured to restrict database operations and prevent unauthorized access to protected collections. These rules ensure that only authenticated users can perform permitted actions within the system.

Input validation was applied throughout the application to reduce the risk of invalid or malformed data being stored in the database. Forms enforce required fields, data type validation, and basic consistency checks before submission.

Error handling mechanisms were implemented using try-catch blocks and asynchronous error management techniques. Database operations, authentication requests, and administrative actions are monitored for potential failures, allowing the application to respond gracefully when errors occur.

User-friendly notifications were integrated using React Hot Toast to provide immediate feedback regarding successful operations, validation issues, and system errors. This approach improves usability while helping users understand the outcome of their actions.

These security and error-handling practices contribute to a more reliable, maintainable, and secure application environment.

4.2 Version Control & Collaboration

4.2.1 Version Control Repository

Version control was managed using Git and GitHub throughout the development process. The repository served as the central platform for storing source code, tracking changes, and facilitating collaboration among team members.

GitHub provided a structured environment for managing project files, monitoring development progress, and maintaining a complete history of modifications. All source code, documentation, configuration files, and project assets were stored within the repository to ensure consistency and accessibility.

The repository enabled team members to work on different features while maintaining synchronization with the latest project version. Version control practices helped reduce conflicts, improve traceability, and support efficient collaboration during development.

Using Git also allowed the team to maintain historical versions of the project, making it possible to review previous implementations, revert changes when necessary, and monitor the evolution of the application over time.

Repository Link:
https://github.com/sohaib-ayman/marketly

4.2.2 Branching Strategy

A feature-based branching approach was followed during the development of Marketly to organize code changes and reduce integration conflicts. Team members worked on separate features and enhancements before merging their work into the main codebase.

The main branch was maintained as the stable version of the project, containing tested and functional code. New features, bug fixes, and improvements were developed independently and reviewed before being integrated into the primary branch.

This workflow allowed multiple team members to contribute simultaneously without affecting the stability of the application. It also simplified testing and validation by isolating changes until they were ready for integration.

The branching strategy improved collaboration, reduced the risk of overwriting code, and provided a structured development process throughout the project lifecycle. By separating development tasks into manageable units, the team was able to maintain code quality while supporting continuous feature development.

4.2.3 Commit History and Documentation

Git commits were used throughout the development process to record project progress and document implemented features. Each commit represented a specific modification, enhancement, bug fix, or system improvement, providing a clear development timeline.

Meaningful commit messages were used to describe the purpose of each change, making it easier to understand project evolution and track completed tasks. This practice improved maintainability by allowing team members to quickly identify when and why specific modifications were introduced.

The commit history also served as a development log, documenting the implementation of major features such as authentication, product management, order processing, role-based access control, backup and restore functionality, and administrative tools.

Version control documentation helped the team review previous changes, identify issues more efficiently, and maintain a reliable record of project development activities. The complete commit history remains available within the repository for future reference and maintenance purposes.

4.3 Deployment & Execution

4.3.1 README File

A comprehensive README file was created to provide clear documentation for the Marketly project. The README serves as the primary reference for developers, evaluators, and future contributors by explaining the project's purpose, architecture, setup process, and available features.

The documentation includes detailed installation instructions, project requirements, configuration steps, execution guidelines, and deployment information. It also provides an overview of the system architecture, technology stack, authentication mechanisms, administrative features, and database integration.

In addition, the README contains information about the project structure, team members, and the deployed application link. This documentation simplifies project setup, improves maintainability, and helps new developers understand the system with minimal onboarding effort.

The README is maintained alongside the source code repository to ensure that project documentation remains synchronized with the latest implementation and feature updates.

4.3.2 System Requirements

The Marketly application requires a modern web development environment and internet connectivity to support communication with Firebase services and other external resources.

Software Requirements

- Node.js (Version 18 or later)
- npm (Node Package Manager)
- Modern Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari)
- Git (for repository cloning and version control)

Dependencies

The project relies on several frontend and backend libraries, including:

- React
- React Router DOM
- Redux Toolkit
- Firebase
- Bootstrap
- CSS Modules
- React Helmet Async
- React Hot Toast
- Formik
- Yup

All required dependencies can be installed automatically using the npm package manager.

Hardware Requirements

The application does not require specialized hardware and can run on standard personal computers with:

- Dual-core processor or higher
- Minimum 4 GB RAM
- Stable internet connection
- Modern web browser support

Browser Compatibility

Marketly is designed to operate on modern browsers that support current JavaScript standards and responsive web technologies. The application has been optimized for desktop and mobile devices to ensure a consistent user experience across different platforms.

4.3.3 Configuration and Execution Guide

A structured configuration and execution process was established to simplify project setup and deployment. The application was designed to be easily installed and executed in a standard web development environment.

Project Setup
The source code can be obtained by cloning the project repository:
git clone https://github.com/sohaib-ayman/marketly

After cloning the repository, the project directory should be opened and all required dependencies installed using npm:
npm install

Firebase Configuration
The application relies on Firebase Authentication and Cloud Firestore services. Before running the project, a Firebase project must be created and configured. Authentication and Firestore services should be enabled, and the Firebase configuration values must be added to the application's configuration file.

Running the Application
Once all dependencies have been installed and Firebase has been configured, the development server can be started using:
npm start

The application will then be accessible through the local development server and can be tested using a modern web browser.

Production Deployment
The production version of Marketly is deployed using Vercel. The deployment platform automatically builds and publishes the latest version of the application whenever updates are pushed to the main branch of the repository.

Deployment Link
https://marketly-nu.vercel.app

This deployment process ensures that the live application remains synchronized with the latest stable version of the source code.

4.3.4 API Documentation

Marketly integrates with several external services to provide authentication, data storage, and product management functionality. The application primarily relies on Firebase services for backend operations and uses external APIs where appropriate.

Firebase Authentication
Firebase Authentication is responsible for user registration, login, logout, session management, and anonymous authentication. The service provides secure user identity verification and supports role-based access control within the application.

Main operations include:

- User Sign Up
- User Login
- User Logout
- Session Persistence
- Anonymous Authentication

Cloud Firestore
Cloud Firestore serves as the primary database for the application. It stores and synchronizes all core data, including products, categories, users, orders, and backup records.

Main collections include:

- Products
- Categories
- Users
- Orders
- Backups

Firestore real-time listeners are used to ensure that data changes are immediately reflected across the application without requiring page refreshes.

Platzi Products API
The Platzi Products API is used exclusively for product seeding purposes. Products can be imported from the external API and stored within Firestore collections, allowing the application to operate independently from the external service after the import process is completed.

Typical usage includes:

- Fetching product data
- Importing products into Firestore
- Initial catalog population

API Communication Flow
The frontend communicates directly with Firebase services through the Firebase SDK. Authentication requests are handled by Firebase Authentication, while all database operations are performed through Cloud Firestore. Product seeding operations additionally interact with the Platzi Products API before storing the retrieved data in Firestore.

This architecture provides a scalable, secure, and real-time data management solution while minimizing dependency on external data sources during normal application operation.

TESTING

Automated Test Dashboard

Test Cases

- Automated Test Toxing ✓
- Automated Test Dashboard ✓
- Automated Test Desige ✓
- Automated Test Dashboard ✓
- Automated Test Dashboard ✓
- Automated Test Verified ✓

PASSED

User Validation

User Loername

Username

Password

Log in

User Authentication Testing

Vsitr Authentication

Log in

Bug Tracking Board

8

Eberrers | Roates | Bisweker 1s

Performance Metrics Charts

Performance Audis

10%

Lighthouse Audit Results

59% 36% 50% 50%

50% 38% 3% 100%

Unit Testing Panel

Test Cases

```
COAIR Eixnopca = » {
  saine[ 1ca.6Sa2ouBeppiito = {
    vrbré('heanon("ynoeuntes/deaf.pona1")
  };

  conts 6n6tS() {
    cuntt groeXOktoaxtont(ins1e1.o8670oax:000 => {
    reGuvte {
      soircle.aetBaetvlotatl£nk));
      relker);
    }));

    contt anaatoskbbs = sxgerrbootelont."dorxBatoe:oon");

    contt gnaaitoe {
    corlaen-neattmvirod{
      gtobalsesn8BoEean("oe2aen");
      getNes.opevebXeata("parshkar");
    };
  };
}
```

Database Validation Checks

Product Name	Status	Astined
Database Validation1	Susrded	-
Database Validation2	Soseded	-
Database Validationlt	Soerded	-
Database Validation2	Susrded	-
Database Validationlt	Soorded	-

Database Validation

Database → Use SQL

Marketly

Home | Clatives | Electronics | Fuuitsuce | Bruus | Miocullanoss | Track Order | owner

Welcome to Marketly

Discover amazing products al greal prices

Shop Now →

Shop by Category

Home | Clatiws | Slextronies | Shoes | Trach Eiviby

Order Processing Test Flow

Test Cases

Test Case → Test Cases → Security Check → Order Processing Test Pioic → Order Processing → Order Processing

Security Check

Security Validation

QA Report

Username

Password

Verified

Verified

Mobile Responsiveness Testing Preview

Browser Compatibility

Browser	✓	✓	✗	✓
Daroarl	✓	✓	✗	✗
Daccelo	✓	✓	✗	✓
Dodbs	✓	✓	✓	✗
Tortter	✓	✓	✗	✗
Voodt	✓	✓	✓	✗

5.1 Test Cases & Test Plan

A structured testing approach was followed to verify the functionality, reliability, and usability of the Marketly platform. The testing process focused on validating core system features, user interactions, administrative operations, and database integration.

The primary objective of testing was to ensure that all system requirements were implemented correctly and that users could interact with the platform without encountering functional issues. Both normal and edge-case scenarios were considered during testing to evaluate system behavior under different conditions.

Test cases were designed to cover the main modules of the application, including authentication, product browsing, shopping cart operations, order processing, user management, product management, and administrative functionality. Each test case included predefined inputs, expected outcomes, and validation criteria to ensure consistent evaluation.

The testing process was conducted throughout development and continued after major feature implementations. This iterative approach helped identify issues early, verify bug fixes, and maintain overall system stability.
The following test cases summarize the most important functional tests performed on the application.

Functional Testing:

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC-01	User Registration	Create a new account using valid credentials	Account is created successfully and user document is added to Firestore	Passed
TC-02	User Login	Login using valid credentials	User is authenticated and redirected to the homepage	Passed
TC-03	Invalid Login	Login using incorrect credentials	Authentication is denied and an error message is displayed	Passed
TC-04	Guest Browsing	Access the platform without logging in	Anonymous user session is created successfully	Passed
TC-05	Add Product to Cart	Click “Add to Cart” on a product	Product is added to the shopping cart	Passed
TC-06	Remove Product from Cart	Remove a product from the cart	Product is removed and cart totals are updated	Passed
TC-07	Cart Persistence	Refresh the page after adding products	Card data remains available	Passed
TC-08	Product Search	Search for an existing product	Matching products are displayed	Passed
TC-09	Category Filtering	Open a category page	Only products belonging to that category are displayed	Passed
TC-10	Product Details	Open a product details page	Complete product information is displayed	Passed

Test Case ID	Test Scenario	Test Steps	Expected Result	Status
TC-11	Place Order	Complete checkout with valid information	Order document is created in Firestore	Passed
TC-12	Order Tracking	Search using a valid order ID	Correct order information is displayed	Passed
TC-13	View Order History	Open My Orders page	User orders are retreived from Firestore	Passed
TC-14	Admin Product Creation	Creating a new product from dashboard	Product appears in Firestore and website catalog	Passed
TC-15	Admin Product Editing	Modify an existing product	Changes are reflected immediately in Firestore	Passed
TC-16	Admin Product Deletion	Delete and existing product	Product is removed successfully	Passed
TC-17	Order Status Update	Change order status from dashboard	Updated status is saved in Firestore	Passed
TC-18	User Management	Remove a user record from dashboard	User document is removed from Firestore	Passed
TC-19	Backup Creation	Create a products backup	Backup data is stored successfully	Passed
TC-20	Back Restoration	Restore products from backup	Products are restored correctly	Passed

Non-Functional Testing:

Test Area	**Validation Method**	**Result**
Performance	Lighthouse Performance Audit	Passed
Accessibility	Lighthouse Accessibility Audit	Passed
SEO	Lighthouse SEO Audit	Passed
Responsive Design	Mobile and Desktop testing	Passed
Browser Compatibility	Chrome, Edge, Firefox Testing	Passed
Database Connectivity	Firestore Integration Testing	Passed
Authentication Security	Firebase Authentication Testing	Passed
Role-Based Access Control	User, Admin, and Owner Validation	Passed

5.2 Automated Testing

Although dedicated unit testing and end-to-end testing frameworks were not implemented during this project, several automated validation and analysis tools were used to evaluate application quality and performance.

Google Lighthouse was used to perform automated audits covering performance, accessibility, best practices, and search engine optimization. These audits provided detailed reports and recommendations that were used to improve the overall quality of the application.

Responsive behavior was validated across different screen sizes to ensure consistent functionality on desktop and mobile devices. Browser compatibility testing was also performed to verify that the application behaved correctly in modern web browsers.

Firebase Authentication and Cloud Firestore integrations were continuously validated through automated service responses and real-time data synchronization checks. This helped ensure that authentication workflows, database operations, and user interactions functioned as expected.

Throughout development, automated build validation provided by the React development environment and Vercel deployment platform was used to detect compilation errors, dependency issues, and deployment failures before releasing updates to the production environment.

These automated validation processes contributed to maintaining application stability, reliability, and overall software quality.

5.3 Bug Reports and Issue Resolution

During the development and testing phases, several issues were identified and resolved to improve system reliability, usability, and overall performance. The following table summarizes the most significant issues encountered throughout the project lifecycle.

Issue ID	Issue Description	Cause	Resolution	Status
BR-01	Firestore access requests were denied after deployment	Expired Firestore security rules	Updated and reconfigured Firestore security rules with appropriate permissions	Resolved
BR-02	Newly added products were not displayed in category pages	Product price exceeded the predefined maximum filter value	Updated filtering logic to support dynamic price ranges	Resolved
BR-03	Product cards displayed inconsistent heights	Product images had different dimentions and aspect ratios	Standardized image rendering and card layout structure	Resolved
BR-04	Order status controls were editable for protected orders	Missing permission restrictions in the dashboard	Implemented role-based validation and protected order handling	Resolved
BR-05	Product management modal exceeded viewport height	Dynamic image fields increased modal content size	Added scrollable modal body and improved layout behavior	Resolved
BR-06	Backup restoration failed due to insufficient permissions	Missing Firestore permissions for backup collections	Updated Firestore security rules and validated access permissions	Resolved
BR-07	Product image management supported only a single image	Initial implementation did not supprot image arrays	Added dynamic image field management for create and edit operations	Resolved

Issue ID	Issue Description	Cause	Resolution	Status
BR-08	Order management interface did not clearly indicate restricted actions	Disabled controls lacked visual distinction	Added dedicated styling for locked controls and permission indicators	Resolved
BR-09	Responsive layout inconsistencies appeared on smaller screens	Cetain dashboard components were not fully optimized for mobile devices	Updated responsive styling and layout rules	Resolved

The bug resolution process was performed iteratively throughout development. Each issue was reproduced, analyzed, corrected, and re-tested to verify that the implemented solution resolved the problem without introducing additional defects. This approach helped improve system stability, user experience, and maintainability before the final deployment phase.

CONCLUSION

Project Goals Achieved

- ✓ Requirements Met
- ✓ System Stability

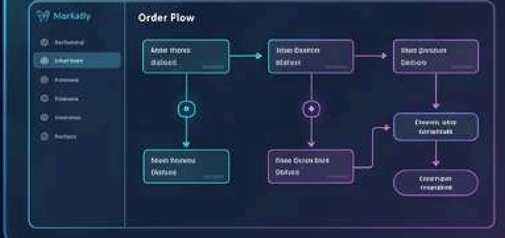
✓ Project Delivered

Planning → Analysis → Design → Implementation → Testing

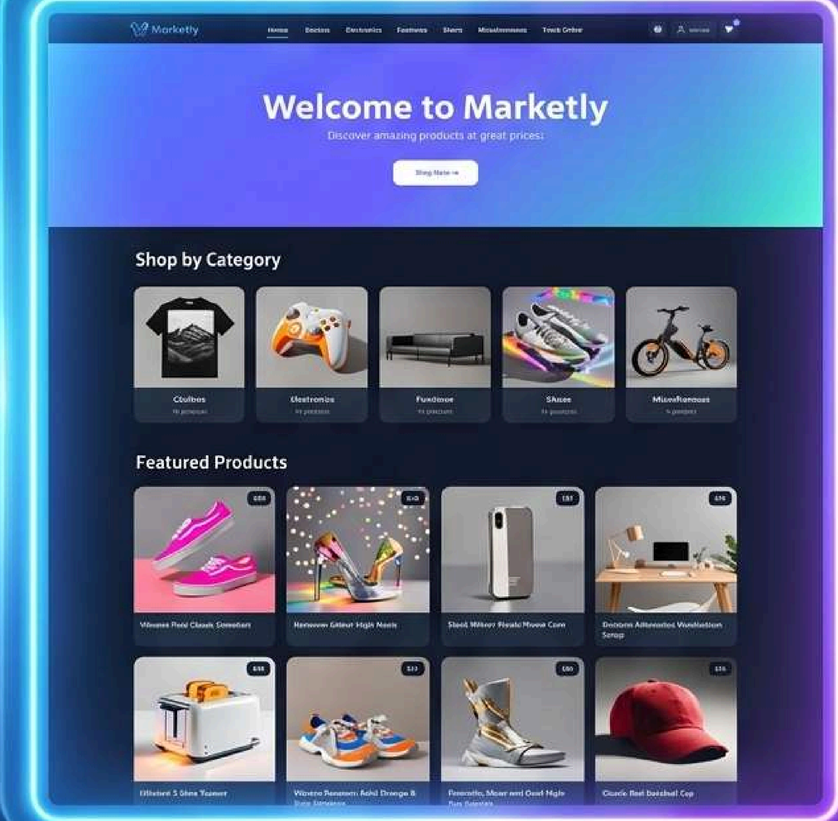
Admin Dashboard Completed

User Authentication Completed

Order Management System



E-Commerce Features Delivered



Performance Optimization Results

Admin Dashboard Completed

Security Validation

Firewall

Encryption

Responsive Design Success

- ✓ Testing Passed
- ✓ Deployment Successful
- ✓ Production Ready

Database Integration Completed

FUTURE VISION

- ✓ Online Payments
- ✓ Product Reviews
- ✓ Product Reviews
- ✓ Advanced Analytics
- ✓ Inventory Management
- ✓ Email Notifications
- ✓ AI Recommendations

The development of Marketly successfully achieved the primary objective of creating a modern and scalable e-commerce platform using React, Firebase Authentication, Cloud Firestore, and Redux Toolkit. The system provides a complete shopping experience that includes product browsing, category navigation, shopping cart management, order processing, and order tracking.

In addition to customer-facing features, the platform includes a comprehensive administrative dashboard that supports product management, user management, order management, backup and restore operations, and product seeding capabilities. Role-based access control was implemented to ensure that administrative functions are securely managed.

Throughout the development process, emphasis was placed on maintainability, responsiveness, security, and real-time data synchronization. Multiple testing and validation activities were performed to ensure system reliability and usability across different devices and environments.

Several challenges were encountered during development, including database security configuration, real-time data synchronization, responsive interface optimization, and administrative permission management. These challenges were successfully addressed through iterative development and testing.

Future enhancements may include online payment integration, advanced analytics dashboards, inventory management, product reviews and ratings, automated email notifications, and expanded reporting capabilities.

Overall, Marketly demonstrates the successful application of modern web development technologies to build a functional, secure, and user-friendly e-commerce solution.

Analysis

Planning

Analysis

Implementation

Testing

Implementation

Testing

Deployment

- ✓ Project Completed
- ✓ Database Connected
- ✓ System Deployed
- ✓ Orders Processing
- ✓ Authentication Ready
- ✓ Security Verified

MARKETLY

Modern E-Commerce Platform

Digital Egypt Pioneers Initiative (DEPI)